最小二乘方法：Python 交互学习

Least-Squares-Methods-in-Python · 中文初版 v0.1

Ancho5515 / Least-Squares-Methods-in-Python

2026 年 11 月 3 日

目录

阅读与运行指南 **1**

0.1 如何阅读 1

0.2 公共初始化代码 1

0.3 最小运行实验 2

0.4 安装、运行与编译 3

0.5 如何判断“学会了” 3

0.6 练习与解答 3

第一章 从求平均到拟合直线 **4**

1.1 平均值是一个优化问题的答案 4

1.2 平方为什么既方便又危险 5

1.3 从一个常数变成一条直线 5

1.4 如何阅读残差图 6

1.5 练习与解答 6

第二章 如何建立最小二乘模型 **7**

2.1 先写出观测方程 7

2.2 “线性”指的是对未知参数线性 7

2.3 单位不同的残差不能直接相加 8

2.4 最小值与可用答案的距离 8

2.5 练习与解答 8

第三章 线性最小二乘的数学原理 **10**

3.1 从目标到正规方程 10

3.2 几何上是在做投影 10

3.3 参数空间与观测空间不要混淆 11

3.4 一个很有用的误差分解 11

3.5 练习与解答 12

第四章 如何可靠地求解线性问题 13

4.1 为什么正规方程可能放大困难 13
4.2 QR：旋转到更容易的坐标 13
4.3 SVD：直接看到哪些方向缺乏信息 13
4.4 秩亏时，返回一个向量不等于参数唯一 14
4.5 练习与解答 15

第五章 噪声、权重与估计可信度 16

5.1 独立误差的权重 16
5.2 相关误差需要整体白化 17
5.3 参数的不确定性 17
5.4 练习与解答 18

第六章 正则化与过拟合 19

6.1 将偏好写进目标 19
6.2 怎样选择惩罚强度 20
6.3 练习与解答 21

第七章 进入非线性最小二乘 22

7.1 残差向量的局部模型 22
7.2 亲手写出指数模型的雅可比 22
7.3 数值微分如何选步长 23
7.4 练习与解答 24

第八章 迭代优化的基本工具 25

8.1 下降方向和步长 25
8.2 为什么最速下降会来回摆动 25
8.3 牛顿法利用曲率 26
8.4 停止规则各自说明什么 27
8.5 练习与解答 27

第九章 Gauss–Newton 方法 28

9.1 推导一步更新 28
9.2 先写一个未经保护的 GN 循环 28

9.3 给方向加上回溯保护 . . . 29
9.4 何时不该继续硬走 GN . . . 30
9.5 练习与解答 . . . 30

第十章 Levenberg–Marquardt 方法 31
10.1 阻尼作用在步长上 . . . 31
10.2 用增益比检验模型 . . . 31
10.3 LM 仍然有边界 . . . 33
10.4 练习与解答 . . . 33

第十一章 信赖域与狗腿法 34
11.1 两个容易计算的候选步 . . . 34
11.2 半径如何调整 . . . 35
11.3 练习与解答 . . . 36

第十二章 让算法适应真实数据 37
12.1 先把参数尺度写清楚 . . . 37
12.2 边界不是裁剪一下就等价 . . . 37
12.3 稳健损失降低异常值支配 . . . 37
12.4 一个有顺序的诊断流程 . . . 38
12.5 练习与解答 . . . 39

第十三章 递推最小二乘 40
13.1 从批量目标到递推 . . . 40
13.2 遗忘使估计能跟踪变化 . . . 41
13.3 与卡尔曼滤波的联系和差别 . . . 42
13.4 练习与解答 . . . 42

第十四章 综合案例：曲线拟合与标定 43
14.1 在拟合前固定实验规则 . . . 43
14.2 可辨识性和局部不确定性 . . . 44
14.3 将结果写成可以复查的结论 . . . 45
14.4 练习与解答 . . . 45

第十五章 综合案例：PWM→XYZ 校正 46
15.1 明确仿真假设 . . . 46

15.2 用实际扰动跨度估计双侧雅可比 47
15.3 低灰平台和不可达目标 49
15.4 练习与解答 49

附录 A 必要数学基础 **50**
A.1 向量、内积与范数 50
A.2 导数、梯度与二阶展开 50
A.3 正定性、特征值与奇异值 50
A.4 概率与协方差 51
A.5 练习与解答 51

附录 B 符号、公式与算法选择 **52**
B.1 符号速查 52
B.2 选择方法前先选择目标 52
B.3 六项交付检查 53
B.4 公式速查 53
B.5 一段可以运行的初步诊断 53

附录 C 进阶方法的入口 **54**
C.1 可分最小二乘：先消去线性参数 54
C.2 割线更新：用一次新观测修正雅可比 55
C.3 混合 LM 与拟牛顿的动机 55
C.4 练习与解答 56

附录 D 参考来源、对应关系与勘误 **57**
D.1 主要来源 57
D.2 与 2004 年原书的对应 57
D.3 如何处理原书已记录的疑点 58
D.4 初版的验证与边界 58
D.5 练习与解答 58

阅读与运行指南

最小二乘把一组不完全一致的观测，变成一个可以比较、可以计算的模型。本书希望你最终能回答四个问题：未知量是什么，为什么这样定义误差，算法每一步在做什么，以及什么时候不应该相信结果。

这是可执行的中文初版教材。建议先读 01—04 章，亲手改动一个观测；再读 05—06 章，观察噪声和正则化；最后进入 07—12 章的迭代方法。13 章适合在线估计，14—15 章把方法放进完整标定流程。数学基础可随时查附录 A。

本书借鉴 Roger R. Labbe 的 *Kalman and Bayesian Filters in Python* 的教学组织方式：用可修改的实验建立直觉，再给出必要推导。非线性部分主要参考 Madsen、Nielsen、Tingleff（2004），章节对应见附录 D。本文的讲解、实验和实现以本仓库为准。

0.1 如何阅读

每章从一个具体问题开始。先在纸上预测图形或数值如何变化，再运行代码。遇到“练习与解答”时可先遮住答案；答案用于自学核对。第一次阅读可以跳过附录 C的进阶方法。

Notebook 中的每幅图均由相邻 Python 代码生成。图内使用英文标签，以便在没有中文绘图字体的系统中复现；图前的中文说明给出变量含义和观察任务。每章都有独立初始化单元，PDF 只在本章显示公共初始化代码。

本书采用列向量数学记号，Python 参数向量用一维数组表示。统一定义残差为“模型减观测”：$r_i(x) = f_i(x) - y_i$，目标为

$$F(x) = \frac{1}{2}\sum_{i=1}^{m} r_i(x)^2 = \frac{1}{2}\|r(x)\|_2^2.$$

原参考书常用“观测减模型”，两者目标相同；使用哪一种，都必须让雅可比与残差的符号一致。

0.2 公共初始化代码

以下代码在每章 Notebook 中都存在。所有示例均可离线运行，不会下载数据。

```
from pathlib import Path
import sys
```

```
root = Path.cwd().resolve()
while not (root / 'lsq').is_dir():
    if root == root.parent:
        raise RuntimeError('Run inside the repository')
    root = root.parent
if str(root) not in sys.path:
    sys.path.insert(0, str(root))
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
from lsq import (finite_difference, whiten, gauss_newton,
                 levenberg_marquardt, dogleg, recursive_update)
np.set_printoptions(precision=5, suppress=True)
plt.rcParams.update({'figure.figsize': (6.4, 3.3), 'font.size': 10,
                     'axes.grid': True, 'savefig.dpi': 130})
```

0.3　最小运行实验

我们先验证环境中的数组计算、求解和绘图。若你把斜率 2 改为 3，拟合值也应该随之改变。

```
t = np.arange(5., dtype=float)
A = np.column_stack([np.ones_like(t), t])
y = 1 + 2*t
x, _, rank, singular = np.linalg.lstsq(A, y, rcond=None)
print('intercept, slope:', x, 'rank:', rank)
assert np.allclose(x, [1, 2])
plt.plot(t, y, 'o', label='data')
plt.plot(t, A@x, label='fit')
plt.xlabel('t'); plt.ylabel('y'); plt.legend(); plt.show()
```

```
intercept, slope: [1. 2.] rank: 2
```

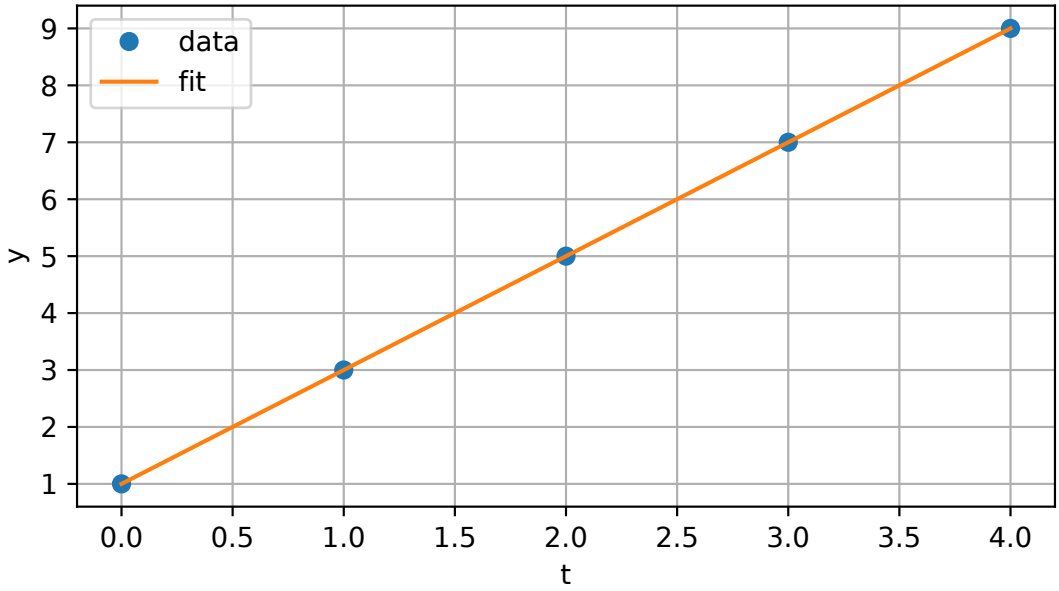


图 1: 环境验证：直线观测与拟合

0.4 安装、运行与编译

在 Python 3.11 或更新版本（建议 3.12）的虚拟环境中，先运行 `python -m pip install -r requirements.txt`。交互阅读还可安装 JupyterLab，并用 `python -m jupyterlab` 打开 `notebooks/`。

本地验证：`python -m unittest discover -s tests -v`。执行全部章节并生成 PDF：`python tools/build.py --execute --pdf`。仅用已保存的执行结果重新排版：`python tools/build.py --pdf`。PDF 编译还需要 Pandoc 和含中文组件的 TeX Live（提供 XeLaTeX 和 latexmk）。完整说明见仓库 README。

生成文件为 `pdf/Least-Squares-Methods-in-Python-zh.pdf`。实际数值、图形和测试记录均提交 Git；构建缓存放在 `.build/`。仓库不配置 Actions 或 Artifact。离线构建的前提是已经安装这些依赖。

0.5 如何判断“学会了”

能调用函数只是第一步。你还需要检查数据和参数的量纲、残差是否有结构、雅可比是否正确、参数是否可辨识，以及失败时的停止原因。“算法停止”不等于“参数是真值”，更不等于“找到全局最优”。

0.6 练习与解答

练习：把上面的时间全部改为零，会发生什么？解答：此时第二列全零，斜率不能由数据确定，矩阵秩从 2 降到 1。程序仍可返回一个最小范数解，但不能据此宣称斜率被测准。第 04 章将正式分析这一现象。

第一章　从求平均到拟合直线

把同一物体称五次，读数略有不同。我们想用一个数 x 代表它的质量。设观测为 $y_1, \dots, y_m$，为什么常常使用平均值，而不是任选一个读数？

1.1　平均值是一个优化问题的答案

考虑 $F(x) = \tfrac{1}{2}\sum_i (x - y_i)^2$。展开或求导得到

$$F'(x) = \sum_i (x - y_i) = m x - \sum_i y_i.$$

令导数为零，得到 $x^\ast = \bar{y}$。因为 $F''(x) = m > 0$，这是唯一的全局极小点。更直接地，利用 $\sum_i (y_i - \bar{y}) = 0$ 可得

$$\sum_i (x - y_i)^2 = \sum_i (\bar{y} - y_i)^2 + m(x - \bar{y})^2.$$

右侧第二项非负，这个等式也给出了不依赖微积分的解释：离开平均值会增加平方误差。

```
y = np.array([10.1, 9.9, 10.2, 10.0, 9.8])
grid = np.linspace(9.6, 10.4, 201)
loss = .5*np.sum((grid[:, None]-y)**2, axis=1)
print('mean:', y.mean(), 'minimum cost:', .5*np.sum((y-y.mean())**2))
assert np.isclose(y.mean(), 10.)
plt.plot(grid, loss); plt.axvline(y.mean(), color='C1', label='mean')
plt.xlabel('candidate mass'); plt.ylabel('half squared error'); plt.legend(); plt.show()
```

```
mean: 10.0 minimum cost: 0.04999999999999964
```

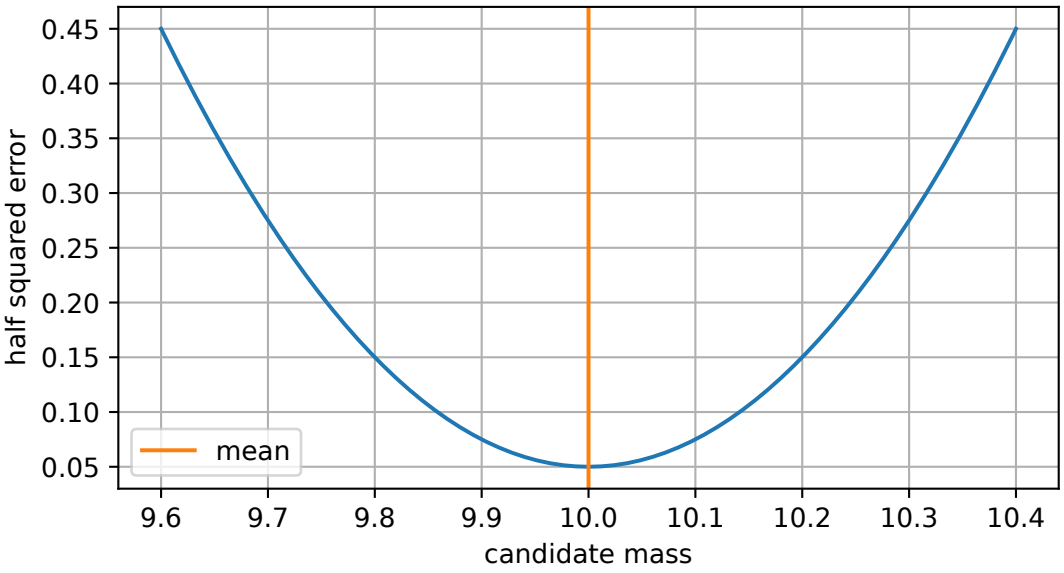


图 1.1: 常数估计的平方损失在平均值处最小

1.2 平方为什么既方便又危险

平方消除了正负抵消，平滑、容易求导；同时，大误差的惩罚增长很快。把一个读数错录为 20，平均值会显著移动。若改为绝对误差和，最优解则是中位数。损失函数反映了我们怎样对待误差，不是数学强制要求平方在任何场景都正确。第 12 章将处理异常值。

1.3 从一个常数变成一条直线

现在测量时间 t_i 和位置 y_i，假设 $f_i(a,b) = a + bt_i$。未知量有截距 a 和斜率 b。令两个偏导为零：

$$\sum_i (a + bt_i - y_i) = 0, \qquad \sum_i t_i(a + bt_i - y_i) = 0.$$

当 t_i 不全相同时，解为

$$b = \frac{\sum_i (t_i - \bar{t})(y_i - \bar{y})}{\sum_i (t_i - \bar{t})^2}, \qquad a = \bar{y} - b\bar{t}.$$

直线必然经过 $(\bar{t}, \bar{y})$。注意：我们最小化的是竖直方向的观测误差，不是点到直线的垂直距离；这等价于把 t 当作足够准确的输入。

```
rng = np.random.default_rng(101)
t = np.linspace(0, 5, 25)
y = 1.2 + 2.0*t + rng.normal(0, .5, len(t))
b = np.sum((t-t.mean())*(y-y.mean())) / np.sum((t-t.mean())**2)
a = y.mean()-b*t.mean()
A = np.column_stack([np.ones_like(t), t])
```

```
reference = np.linalg.lstsq(A, y, rcond=None)[0]
print('hand formula:', [a, b], 'NumPy:', reference)
assert np.allclose([a, b], reference)
fig, ax = plt.subplots(1, 2, figsize=(7, 3))
ax[0].scatter(t, y); ax[0].plot(t, a+b*t, color='C1')
ax[0].set(xlabel='time', ylabel='position')
ax[1].stem(t, a+b*t-y); ax[1].set(xlabel='time', ylabel='residual')
plt.tight_layout(); plt.show()
```

```
hand formula: [np.float64(1.2468646986979914), np.float64(1.9670997346181176)] NumPy: [1.24686 1.9671 ]
```

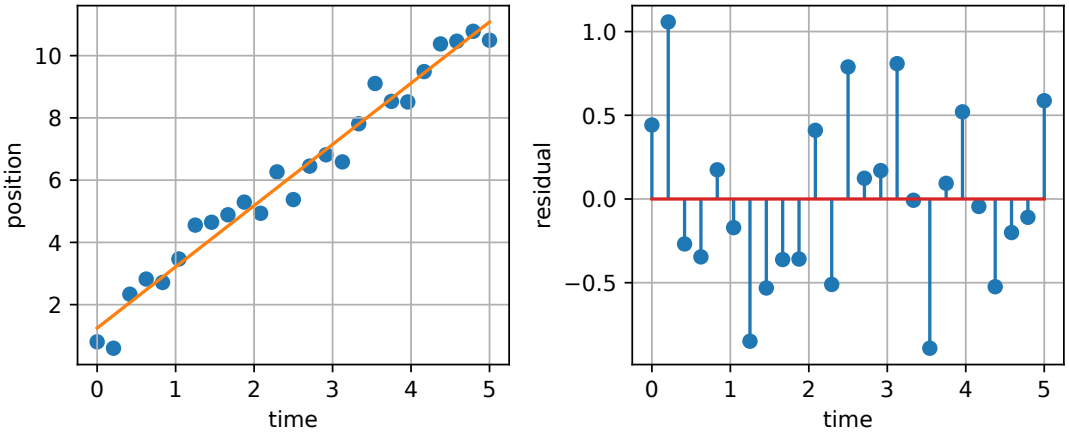


图 1.2: 直线拟合及其残差

1.4 如何阅读残差图

如果残差随时间弯曲，可能是直线模型遗漏了加速度；如果后半段波动明显增大，可能需要不同权重。一个很小的总误差并不能替代残差诊断。对含截距的普通线性最小二乘，残差之和应近似为零，这是可执行的自检。

1.5 练习与解答

1. 所有 y_i 加 3，斜率和截距如何改变？解答：中心化后的 $y_i - \bar{y}$ 不变，斜率不变，截距加 3。
2. 把秒改为毫秒，斜率数值如何改变？解答：新时间是原来的 1000 倍，斜率为原来的 $1/1000$，物理速度没有改变。
3. 将一个数据点增加 8，观察斜率变化，再比较它位于时间区间中间和两端的情况。解答：两端点通常有更大的杠杆作用；异常值影响不仅由残差大小决定，还与输入位置有关。

本章结论：最小二乘给出的是在指定模型和误差定义下最一致的参数；它不能自动替你验证模型假设。

第二章　如何建立最小二乘模型

同一组下降曲线，可以用直线、指数或高阶多项式去拟合。算法会认真地完成你给它的目标，哪怕目标建立错了。本章先把“观测、输入、参数和误差”分清楚。

2.1　先写出观测方程

一个常见模型是 $y_i = f(t_i, x) + \varepsilon_i$。$t_i$ 是已知输入，y_i 是测量结果，$x \in \mathbb{R}^n$ 是待估参数。将所有模型值减去观测，构成 $r(x) \in \mathbb{R}^m$。

$$r_i(x) = f(t_i, x) - y_i, \qquad x^{*} = \arg\min_{x} \frac{1}{2}\|r(x)\|^2.$$

通常希望 $m \geq n$，但观测数量够多并不能保证参数唯一可辨。若两列参数影响几乎相同，增加重复观测也可能无济于事。参数是否可辨识取决于模型对它们的不同响应。

2.2　“线性”指的是对未知参数线性

$a + bt + ct^2$ 对 t 是曲线，但对未知的 (a, b, c) 是线性的。相反，$a\exp(-kt) + c$ 对 k 是非线性的。若 k 已知，只估 a, c，问题又变成线性。这一区分决定了能否一次线性求解，还是需要迭代。

```
t = np.linspace(0, 4, 35)
rng = np.random.default_rng(102)
truth = np.array([2.5, .8, .4])
def model(p):
    a, k, c = p
    return a*np.exp(-k*t)+c
y = model(truth)+rng.normal(0, .04, len(t))
def residual(p):
    return model(p)-y
fit = least_squares(residual, [2., .5, .2], bounds=([0, 0, -np.inf], np.inf))
print('truth:', truth, 'estimate:', fit.x)
print('residual shape:', residual(fit.x).shape, 'parameter shape:', fit.x.shape)
plt.scatter(t, y, label='measurements')
plt.plot(t, model(fit.x), color='C1', label='exponential fit')
plt.xlabel('time'); plt.ylabel('response'); plt.legend(); plt.show()
```

```
truth: [2.5 0.8 0.4] estimate: [2.51287 0.82961 0.42906]
```

```
residual shape: (35,) parameter shape: (3,)
```

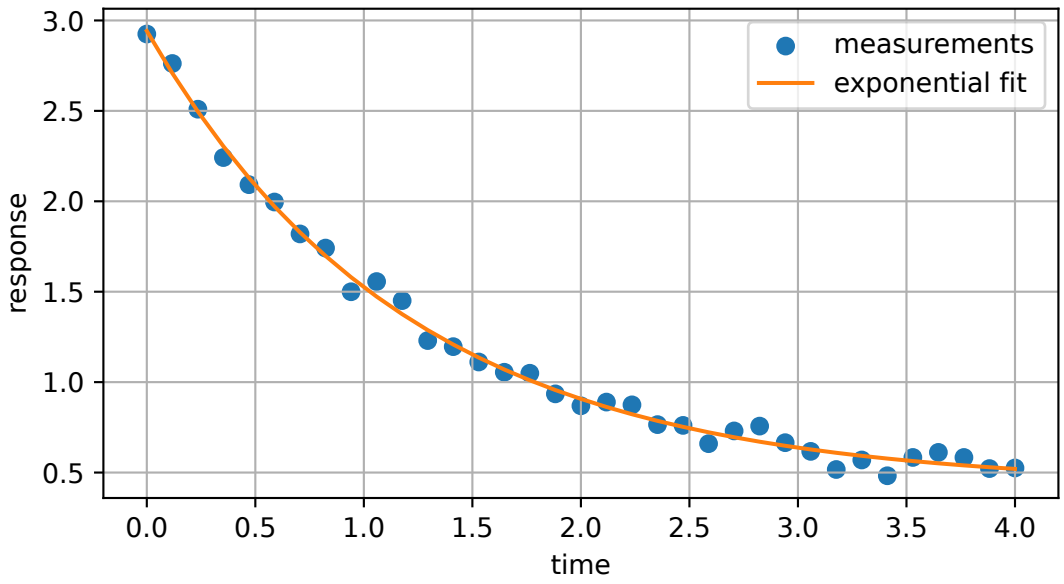


图 2.1：指数模型拟合合成衰减观测

2.3 单位不同的残差不能直接相加

若一个传感器给出毫米，另一个给出伏特，直接把两个残差平方相加，等于用单位选择隐含地决定谁更重要。常用做法是按噪声标准差归一化：$\tilde{r}_i = r_i / \sigma_i$。这既统一量纲，也让高噪声观测少一些影响。相关误差需要整体白化，见第 05 章。

不要通过对 $y - c$ 取对数，未经说明就把指数模型变成直线：$\log(y - c)$ 会改变误差分布和权重，而且当 $y \le c$ 时根本没有实数定义。变换模型和变换损失是两件事。

2.4 最小值与可用答案的距离

残差小，可能是模型合理，也可能是参数太多。参数拟合得好，还可能在观测区间外预测很差。应该分别问：训练数据解释得怎样？未参与拟合的数据预测怎样？参数的物理范围是否合理？最终控制目标是否可达？

2.5 练习与解答

练习：$a\sin(t) + b\cos(t)$、$\sin(at + b)$、$a/(1 + bt)$ 中哪些是线性最小二乘模型？解答：只有第一个对未知参数线性；其设计矩阵两列分别是 $\sin(t_i)$ 和 $\cos(t_i)$。

练习：把残差从 $f - y$ 改为 $y - f$，但仍使用原来的雅可比会怎样？解答：残差与导数符号不一致，梯度方向反转，算法可能不断拒绝步长。必须同时将雅可比乘 -1。

练习：令全部 $t_i = 0$，上面的指数模型能估计哪些参数？解答：只能估计 $a + c$；k 对预测完全没有作用，a 和 c 也无法分开。

第三章　线性最小二乘的数学原理

第 01 章的两条偏导方程，可以写成一个对任意参数数量都适用的矩阵形式。设 $A \in \mathbb{R}^{m\times n}$，每行描述一次观测对参数的线性响应，模型为 Ax。

3.1　从目标到正规方程

$$F(x)=\frac{1}{2}(Ax-y)^T(Ax-y),\qquad \nabla F(x)=A^T(Ax-y).$$

极小点满足 $A^TAx=A^Ty$，这就是正规方程。若 A 满列秩，A^TA 对称正定，解唯一；若秩亏，拟合值仍唯一，但不同参数可给出同一拟合值。

实际编程不推荐显式计算 $(A^TA)^{-1}$。正规方程用于理解数学，数值求解优先使用 QR/SVD，第 04 章会说明原因。

3.2　几何上是在做投影

Ax 只能落在 A 的列空间里。最优拟合 $\hat{y}=Ax^*$ 是 y 在该空间上的正交投影，因此残差 $r=\hat{y}-y$ 与每一列正交，即 $A^Tr=0$。这也说明：每增加一列，允许的拟合空间变大，训练误差不会上升，但预测误差未必下降。

```
v = np.array([1., 2.]); y = np.array([2., .5])
x = (v@y)/(v@v)
projected = x*v
print('coefficient:', x, 'orthogonality:', v@(projected-y))
assert np.isclose(v@(projected-y), 0.)
plt.plot([0, 1.3], [0, 2.6], color='gray', label='column space')
plt.arrow(0, 0, *y, width=.02, color='C0', length_includes_head=True)
plt.arrow(0, 0, *projected, width=.02, color='C1', length_includes_head=True)
plt.plot([y[0], projected[0]], [y[1], projected[1]], '--', color='C2')
plt.axis('equal'); plt.xlabel('observation 1'); plt.ylabel('observation 2')
plt.legend(); plt.show()
```

```
coefficient: 0.6 orthogonality: 0.0
```

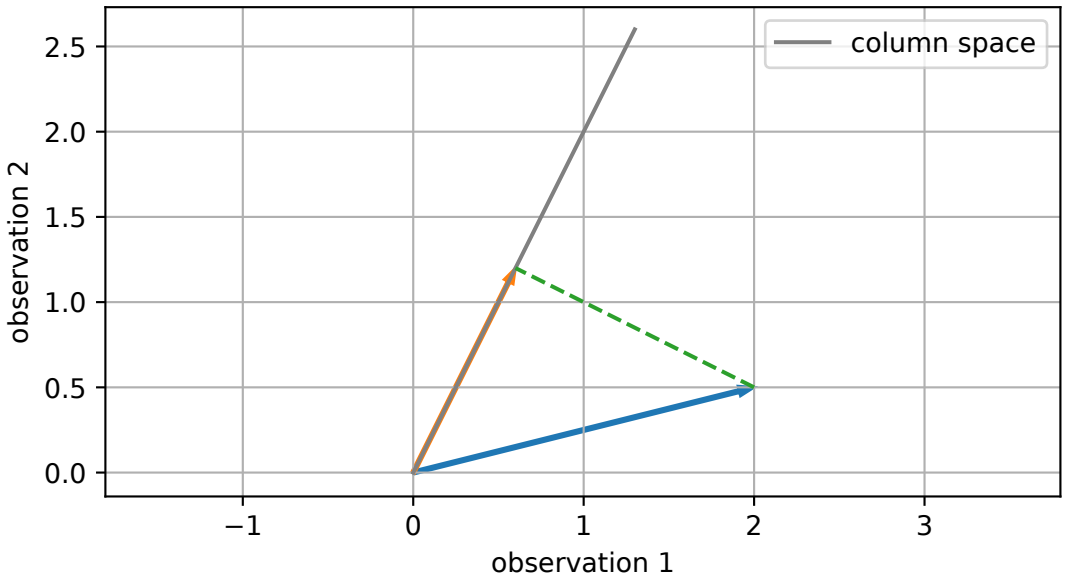


图 3.1: 观测向量在一维列空间上的正交投影

3.3 参数空间与观测空间不要混淆

上图中的坐标轴代表两次观测，而不是两个参数；只有一个参数 x，所以允许的拟合结果是一条直线。对于 m 次观测、n 个独立参数，列空间位于 m 维观测空间中，维数为 n。

满列秩时投影矩阵可写为 $H = A(A^TA)^{-1}A^T$，满足 $H^T = H$、$H^2 = H$。这是数学表达式；下面使用 QR 构造相同投影，避免显式求逆。

```
t = np.arange(6., dtype=float)
A = np.column_stack([np.ones_like(t), t])
y = np.array([1., 2.8, 5.2, 7.1, 8.9, 11.2])
Q, R = np.linalg.qr(A, mode='reduced')
x = np.linalg.solve(R, Q.T@y)
H = Q@Q.T
r = A@x-y
print('parameters:', x, 'A.T @ r:', A.T@r)
print('training residual norm:', np.linalg.norm(r))
assert np.allclose(A.T@r, 0., atol=1e-12)
assert np.allclose(H@H, H)
assert np.allclose(H@y, A@x)
```

```
parameters: [0.94762 2.03429] A.T @ r: [0. 0.]
training residual norm: 0.33580039422535635
```

3.4 一个很有用的误差分解

对任意参数 x，$A(x - x^*)$ 位于列空间内，与最优残差正交。因此

$$\|Ax - y\|^2 = \|Ax^* - y\|^2 + \|A(x - x^*)\|^2.$$

它证明了任何满足正规方程的 x^* 都是全局极小点。若 $Az = 0$ 有非零解，则 $x^* + z$ 给出相同目标值，唯一性就消失了。

3.5 练习与解答

练习：含截距的直线拟合为什么残差和为零？解答：A 的第一列是全 1 向量，正交条件的第一项就是 $\sum_i r_i = 0$。

练习：不含截距时还一定成立吗？解答：不成立；此时全 1 向量未必在列空间里。

练习：把 A 的一列复制一遍，训练残差会下降吗？解答：不会，列空间没有变，只是参数表示不再唯一。代码中观察矩阵秩及 SVD 可验证这一点。

第四章　如何可靠地求解线性问题

同一个数学问题，不同计算方式会有不同误差。浮点运算只能保留有限位数；如果数据稍变就让参数大幅变化，即使算法没有程序错误，结果也可能不可靠。

4.1　为什么正规方程可能放大困难

满列秩时，在 2 范数下 $\kappa_2(A^TA)=\kappa_2(A)^2$。例如 A 的条件数为 10^8，形成 A^TA 后条件数可达 10^{16}，接近双精度可分辨的极限。显式求逆会增加运算和误差，并不帮助解决问题。

4.2　QR：旋转到更容易的坐标

薄 QR 分解 $A=QR$ 满足 $Q^TQ=I$。目标分解为 $\|Rx-Q^Ty\|^2$ 和与 x 无关的正交分量，因而解上三角方程 $Rx=Q^Ty$ 即可。满列秩情况下，QR 避免把条件数平方。

4.3　SVD：直接看到哪些方向缺乏信息

$$A=U\Sigma V^T,\qquad x^*=\sum_{\sigma_j>\tau}\frac{u_j^Ty}{\sigma_j}v_j.$$

较小的奇异值会放大该方向上的数据误差。阈值 τ 区分数值上可分辨的方向；截断也意味着主动放弃某些参数方向的信息。NumPy 的`lstsq`会同时返回秩和奇异值，应当一起查看。

```
t = 1 + .02*np.linspace(-1, 1, 40)
A = np.vander(t, 5, increasing=True)
truth = np.array([1., -2, 3, -.5, .2]); y = A@truth
Q, R = np.linalg.qr(A, mode='reduced')
x_qr = np.linalg.solve(R, Q.T@y)
x_svd, _, rank, s = np.linalg.lstsq(A, y, rcond=None)
try:
    x_normal = np.linalg.solve(A.T@A, A.T@y)
    normal_error = np.linalg.norm(x_normal-truth)
except np.linalg.LinAlgError:
    normal_error = np.inf
print('condition number:', np.linalg.cond(A), 'rank:', rank)
print('parameter error, normal / QR / SVD:', normal_error,
```

```
      np.linalg.norm(x_qr-truth), np.linalg.norm(x_svd-truth))
plt.semilogy(np.arange(1, len(s)+1), s, 'o-')
plt.xlabel('singular direction'); plt.ylabel('singular value'); plt.show()
```

```
condition number: 1400664882.8835294 rank: 5
parameter error, normal / QR / SVD: 2.537145251330903 9.232546032182727e-08 2.271433815690113e-08
```

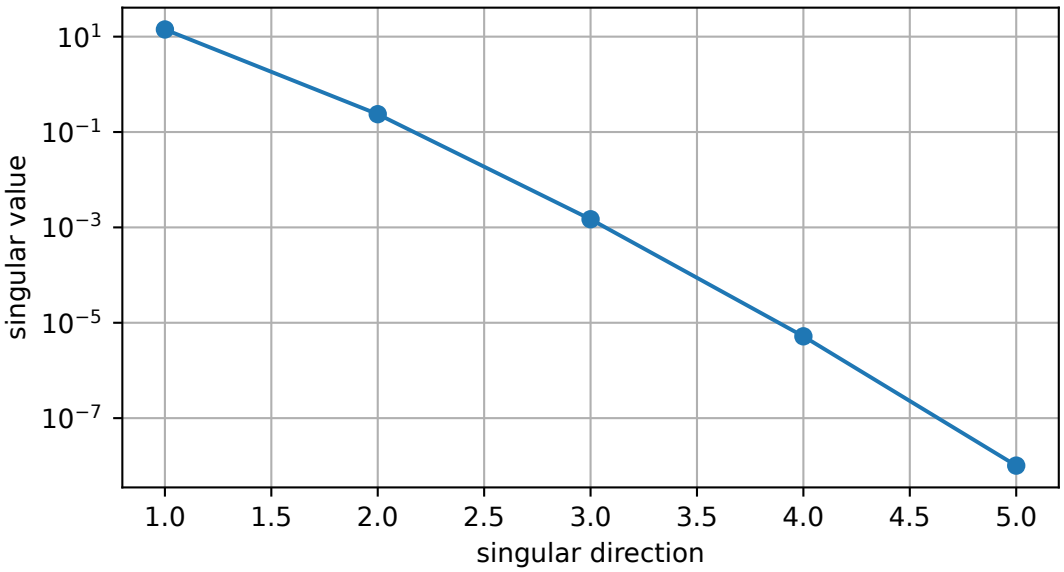


图 4.1: 病态设计矩阵的奇异值跨度

不同 BLAS 实现可能让上面最后几位不同；不要把某台机器上的具体误差比当成数学定理。应观察的是奇异值跨度、形成正规方程的危险，以及参数误差与残差大小的差别。

4.4 秩亏时，返回一个向量不等于参数唯一

当 A 两列完全相同，数据只能约束两参数的和。SVD 可给出欧氏范数最小的解，但这只是选择一个表示的规则，不能凭空创造可辨识性。

```
A = np.ones((4, 2)); y = np.array([1., 2., 2., 3.])
x, residual_summary, rank, s = np.linalg.lstsq(A, y, rcond=None)
print('minimum-norm x:', x, 'rank:', rank, 'summary:', residual_summary)
print('explicit squared residual:', np.sum((A@x-y)**2))
assert np.allclose(x, [1, 1])
assert np.allclose(A@x, A@(x+np.array([5., -5.])))
```

```
minimum-norm x: [1. 1.] rank: 1 summary: []
explicit squared residual: 2.0
```

`lstsq`返回的`residuals`在秩亏或 $m \le n$ 等条件下可能为空；空数组不表示误差为零，应直接计算 $Ax-y$。这是一个容易在实际程序里误判的接口细节。

4.5 练习与解答

练习：将时间 t 中心化并缩放到 $[-1,1]$，再构建多项式矩阵，比较条件数。解答：这常显著改善列的尺度和相关性，但它改变了系数的坐标表示；比较预测曲线比直接比较系数更合适。

练习：最小范数解是否一定最符合物理？解答：不一定，它依赖参数单位和坐标选择。物理先验应通过明确的约束或正则化表达，见第 06 章。

第五章　噪声、权重与估计可信度

若前半组观测精确到 0.1，后半组只能精确到 1.0，等权拟合会给较差观测过多影响。权重的目的，是把不同可靠程度的信息放到同一尺度，而不是把“不喜欢的数据”随意压低。

5.1　独立误差的权重

假设 $y = Ax_{\text{true}} + \varepsilon$，且 $\operatorname{Cov}(\varepsilon) = C = \operatorname{diag}(\sigma_i^2)$。加权目标是

$$F(x) = \frac{1}{2}\sum_{i} \left(\frac{a_i^{T} x - y_i}{\sigma_i} \right)^2 .$$

权重为 $w_i = 1/\sigma_i^2$，实际缩放残差和设计矩阵时乘的是 $\sqrt{w_i} = 1/\sigma_i$。若误差是独立高斯，这同时是最大似然估计；高斯假设不是“能用最小二乘”的必要条件。

```
rng = np.random.default_rng(105)
t = np.linspace(0, 5, 40); A = np.column_stack([np.ones_like(t), t])
sigma = np.where(t < 2.5, .12, .9)
truth = np.array([1., 2.]); y = A@truth+rng.normal(size=len(t))*sigma
x_ols = np.linalg.lstsq(A, y, rcond=None)[0]
Aw, yw = A/sigma[:, None], y/sigma
x_wls = np.linalg.lstsq(Aw, yw, rcond=None)[0]
print('OLS:', x_ols, 'WLS:', x_wls)
plt.errorbar(t, y, yerr=sigma, fmt='.', alpha=.6, label='one sigma')
plt.plot(t, A@x_ols, label='OLS'); plt.plot(t, A@x_wls, label='WLS')
plt.xlabel('t'); plt.ylabel('response'); plt.legend(); plt.show()
```

```
OLS: [1.02546 1.97647] WLS: [1.12019 1.90625]
```

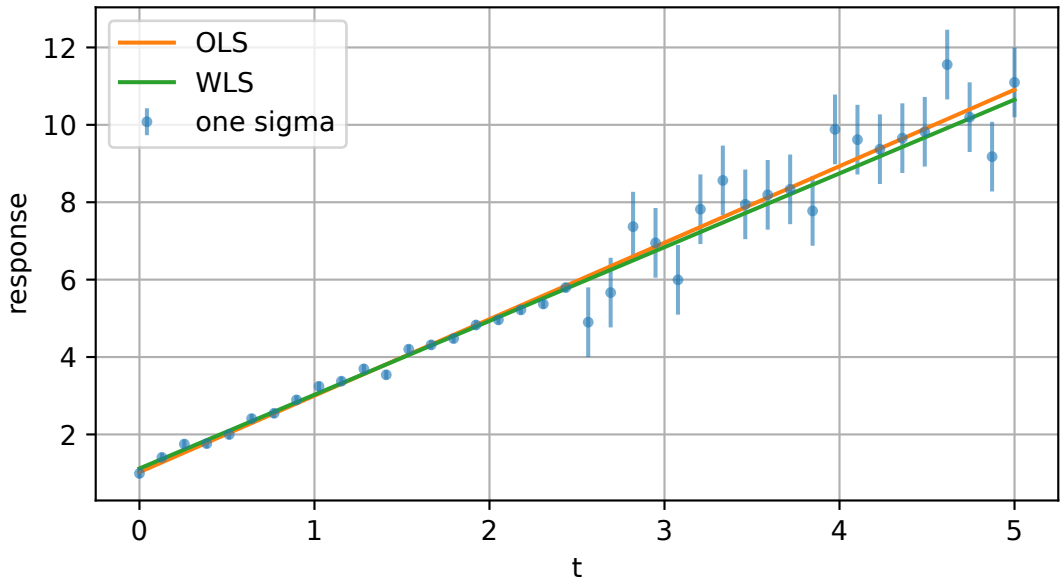


图 5.1: 异方差观测下的普通与加权拟合

5.2 相关误差需要整体白化

一般正定协方差 $C = LL^T$，令 $\tilde{A} = L^{-1}A$、$\tilde{y} = L^{-1}y$。用三角求解得到它们，再做普通最小二乘。因为

$$\|L^{-1}(Ax - y)\|^2 = (Ax - y)^T C^{-1}(Ax - y),$$

这个方向必须是 L^{-1}，不能误用 L^{-T}。实现中无需形成 C^{-1}。

5.3 参数的不确定性

满列秩、线性模型正确且 C 确知时，参数协方差为

$$\mathrm{Cov}(\hat{x}) = (A^T C^{-1} A)^{-1}.$$

若只知道共同的未知噪声尺度 σ^2，通常用 $\hat{\sigma}^2 = \|r\|^2/(m-n)$ 估计，要求 $m > n$ 且模型满秩。不要对已知绝对噪声的情况再次无条件乘这个尺度。

```
C = np.diag(sigma**2)
Aw2, yw2 = whiten(A, y, C)
assert np.allclose(Aw2, Aw)
cov_x = np.linalg.solve(Aw.T@Aw, np.eye(2))
draws = np.array([np.linalg.lstsq(Aw, (A@truth+rng.normal(size=len(t))*sigma)/sigma,
                  rcond=None)[0] for _ in range(600)])
print('predicted standard errors:', np.sqrt(np.diag(cov_x)))
print('Monte Carlo standard errors:', draws.std(axis=0, ddof=1))
```

```
plt.scatter(draws[:, 0], draws[:, 1], s=5, alpha=.3)
plt.plot(*truth, 'rx', markersize=10); plt.xlabel('intercept'); plt.ylabel('slope')
plt.show()
```

```
predicted standard errors: [0.04918 0.03276]
Monte Carlo standard errors: [0.04694 0.03247]
```

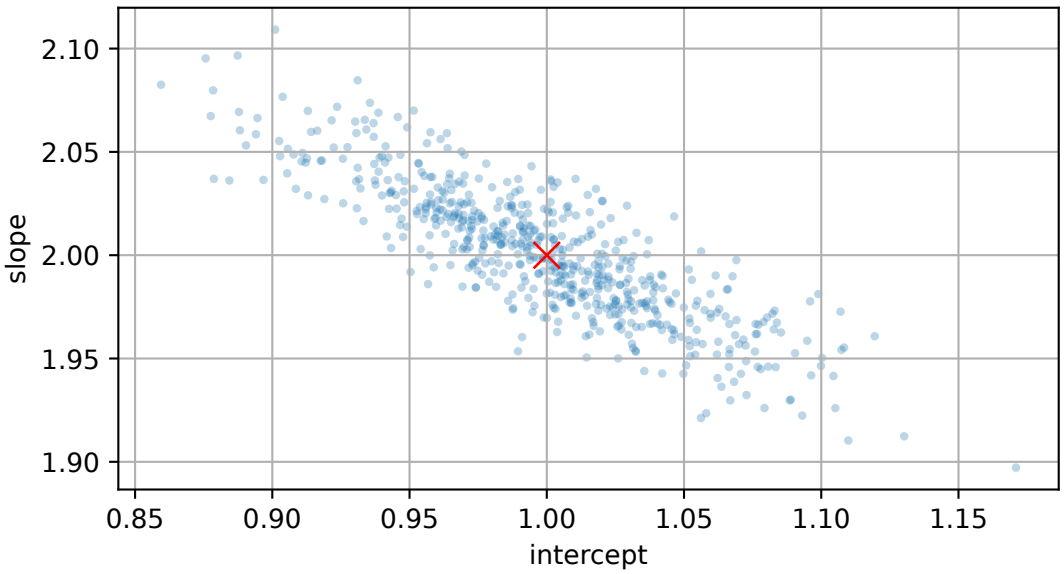


图 5.2: 重复仿真中的截距和斜率估计分布

散点云的倾斜表示参数误差相关。斜率稍大、截距稍小，可能仍拟合同一批数据。每个参数的单独误差条不能完整描述这种关系。

正态近似下，已知协方差时可用 $\hat{x}_j \pm 1.96\,\mathrm{SE}_j$ 给出边际 95% 区间；未知尺度、线性高斯模型下应使用自由度 $m-n$ 的 t 分布。非线性、边界或弱可辨识问题中，这些区间可能很不准确。

5.4 练习与解答

练习：所有标准差乘 2，估计值和协方差如何改变？解答：所有权重统一除 4，最优参数不变，参数协方差乘 4，标准误乘 2。

练习：两个完全相同且噪声完全相关的观测，是否提供双倍信息？解答：不是，完全相关时协方差奇异，不能直接使用本章正定白化。应合并观测或明确处理其有效子空间。

练习：用 600 次仿真的单次结果要求误差标准差与理论值完全相等，合理吗？解答：不合理；蒙特卡洛估计本身有采样误差，应随仿真次数增加检查趋势。

第六章　正则化与过拟合

用 16 阶多项式穿过十几个点，训练误差可能很小，区间边缘却猛烈摆动。最小二乘只看你提供的目标，无法自动知道你更喜欢平滑、简单或物理合理的解释。

6.1　将偏好写进目标

岭型正则化使用

$$F_\lambda(x) = \frac{1}{2}\|Ax - y\|^2 + \frac{\lambda}{2}\|Dx\|^2.$$

$D = I$ 惩罚参数长度；若不希望惩罚截距，可令 $D = \mathrm{diag}(0, 1, \ldots, 1)$。它等价于一个增广最小二乘问题：

$$\min_x \left\| \begin{bmatrix} A \\ \sqrt{\lambda} D \end{bmatrix} x - \begin{bmatrix} y \\ 0 \end{bmatrix} \right\|^2 .$$

正则化引入偏差，以换取对噪声的较低敏感度。D 和参数单位很重要：如果单位改变却不调整惩罚，表达的先验也随之改变。

```
rng = np.random.default_rng(106)
f = lambda t: np.sin(np.pi*t)
t = np.linspace(-1, 1, 18); y = f(t)+rng.normal(0, .18, len(t))
tv = rng.uniform(-1, 1, 70); yv = f(tv)+rng.normal(0, .18, len(tv))
degree = 14
design = lambda t: np.vander(t, degree+1, increasing=True)
A, V = design(t), design(tv)
D = np.diag(np.r_[0., np.ones(degree)])
def ridge(lam):
    return np.linalg.lstsq(np.vstack([A, np.sqrt(lam)*D]),
                           np.r_[y, np.zeros(degree+1)], rcond=None)[0]
lambdas = np.logspace(-7, 1, 45)
validation = np.array([np.mean((V@ridge(l)-yv)**2) for l in lambdas])
best = lambdas[np.argmin(validation)]
plain = np.linalg.lstsq(A, y, rcond=None)[0]; regularized = ridge(best)
grid = np.linspace(-1, 1, 300)
print('chosen lambda:', best, 'coefficient norms:', np.linalg.norm(plain),
      np.linalg.norm(regularized))
fig, ax = plt.subplots(1, 2, figsize=(7, 3))
```

```
ax[0].scatter(t, y, s=12); ax[0].plot(grid, design(grid)@plain, label='OLS')
ax[0].plot(grid, design(grid)@regularized, label='ridge')
ax[0].plot(grid, f(grid), '--', label='truth'); ax[0].set_ylim(-2, 2)
ax[0].set(xlabel='t', ylabel='response'); ax[0].legend()
ax[1].semilogx(lambdas, validation); ax[1].set(xlabel='lambda', ylabel='validation MSE')
plt.tight_layout(); plt.show()
```

```
chosen lambda: 0.00043287612810830614 coefficient norms: 3761.5230070368407 6.291813881254787
```

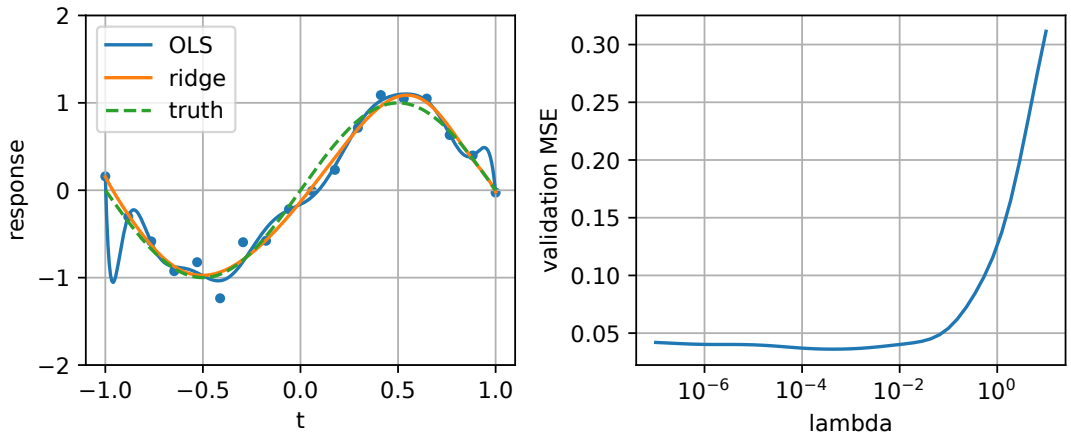


图 6.1: 正则化曲线与独立验证误差

6.2 怎样选择惩罚强度

上例使用独立验证集选 λ。一旦你根据验证结果挑选参数，它就不再是完全未使用的测试集。最终性能应在第三份独立数据上评估；数据少时可用交叉验证，并将调参与最终评估分开。

在 $D = I$ 时，SVD 中的每个方向由 $1/\sigma_j$ 改为 $\sigma_j/(\sigma_j^2 + \lambda)$。小奇异值方向被压低，这解释了稳定性如何改善。它并不保证所有真参数或所有数据集都得到更小误差。

高斯噪声与高斯先验下，正则化还可解释为最大后验估计。若噪声方差为 σ^2、零均值先验方差为 τ^2，在未白化目标的写法下 $\lambda = \sigma^2/\tau^2$。要先固定目标的尺度，再比较不同文献里的 λ。

```
tt = rng.uniform(-1, 1, 200)
yt = f(tt)+rng.normal(0, .18, len(tt))
print('independent test MSE, OLS:', np.mean((design(tt)@plain-yt)**2))
print('independent test MSE, ridge:', np.mean((design(tt)@regularized-yt)**2))
assert np.all(np.isfinite(regularized))
```

```
independent test MSE, OLS: 0.06030827732864676
independent test MSE, ridge: 0.03305857997927622
```

6.3 练习与解答

练习：令 $\lambda \to \infty$，上例的曲线趋向什么？解答：非截距系数被压向零，留下一个近似常数模型；由于截距未受罚，它不必趋向零。

练习：增加一个参数后，带正则化的纯训练残差一定下降吗？解答：若比较的是各自正则化目标的最优解，纯训练残差未必单调；“增加列空间使训练残差不升”针对的是未加额外惩罚的普通最小二乘。

练习：正则化和 LM 阻尼看起来都增加 λI，是否相同？解答：正则化改变最终要解的目标；第 10 章LM 阻尼主要限制当前局部迭代步，通常随迭代调整，二者用途不同。

第七章　进入非线性最小二乘

直线拟合可以一次求解，指数衰减的速率却出现在指数里。我们的办法不是寻找一个万能闭式公式，而是在当前参数附近，把曲线暂时近似成直线，再决定走多远。

7.1　残差向量的局部模型

设 $r:\mathbb{R}^n\to\mathbb{R}^m$ 连续可微，其雅可比为 $J_{ij}=\partial r_i/\partial x_j$。一个小步长 h 给出

$$r(x+h)\approx r(x)+J(x)h.$$

因此局部目标模型是 $q(h)=\tfrac12\|r+Jh\|^2$。它是关于 h 的线性最小二乘，即使原问题关于 x 是非线性的。

目标的梯度和精确 Hessian 分别为

$$g=J^Tr,\qquad H=J^TJ+\sum_{i=1}^{m}r_i\,\nabla^2 r_i.$$

后面一项包含残差的二阶导数。残差很小或模型在附近近似线性时，它可能较小；若残差很大，就不能未经检查把它忽略。

7.2　亲手写出指数模型的雅可比

对 $f(t;x)=a\exp(-kt)+c$，参数顺序固定为 $x=(a,k,c)^T$：

$$J_i=\begin{bmatrix}e^{-kt_i} & -at_ie^{-kt_i} & 1\end{bmatrix}.$$

读清维度：每一行是一条观测，每一列是一个参数。最常见的错误是把 $m\times n$ 转成 $n\times m$，或忘记残差符号。

```
t = np.linspace(0, 4, 30)
p = np.array([2.5, .8, .4])
def model(p):
    a, k, c = p
    return a*np.exp(-k*t)+c
def jac(p):
```

```
    a, k, c = p
    e = np.exp(-k*t)
    return np.column_stack([e, -a*t*e, np.ones_like(t)])
J = jac(p)
numerical = finite_difference(model, p)
print('Jacobian shape:', J.shape)
print('largest derivative difference:', np.max(np.abs(J-numerical)))
assert np.allclose(J, numerical, rtol=1e-7, atol=1e-8)
direction = np.array([.3, -.2, .1])
steps = np.logspace(-5, -1, 12)
errors = [np.linalg.norm(model(p+h*direction)-model(p)-h*J@direction) for h in steps]
plt.loglog(steps, errors, 'o-', label='linearization error')
plt.loglog(steps, errors[-1]*(steps/steps[-1])**2, '--', label='quadratic slope')
plt.xlabel('step length'); plt.ylabel('error norm'); plt.legend(); plt.show()
```

```
Jacobian shape: (30, 3)
largest derivative difference: 5.2015725060527984e-11
```

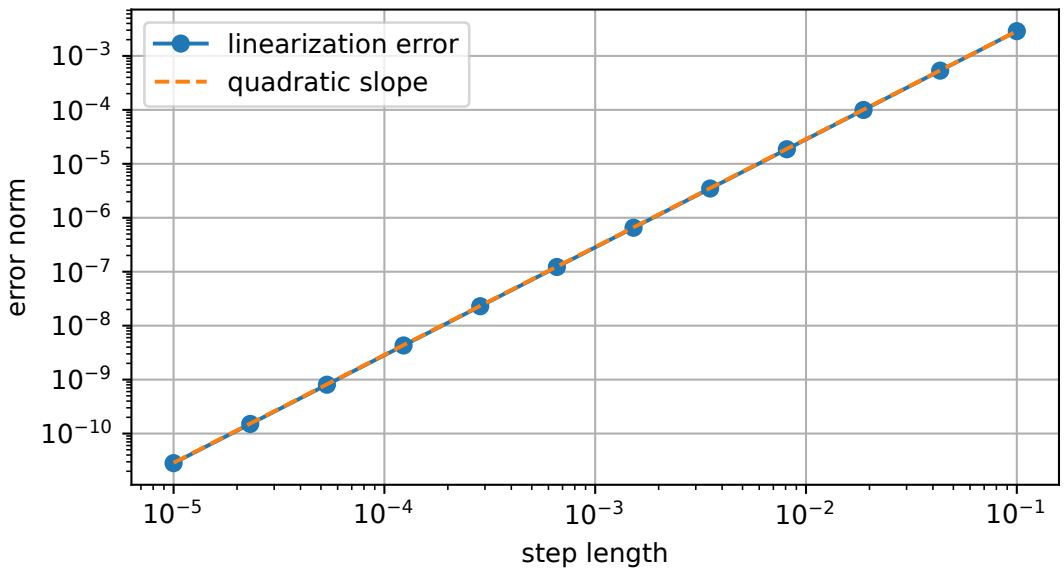


图 7.1: 局部线性化误差随步长变化

图中小步长范围的误差约按 h^2 增长，这检验了线性化。步长太小时，浮点舍入会破坏这个规律。不是扰动越小，数值导数就一定越准确。

7.3 数值微分如何选步长

中心差分使用

$$J_{:,j} \approx \frac{r(x+\delta_j e_j) - r(x-\delta_j e_j)}{2\delta_j}.$$

对足够光滑且无测量噪声的函数，截断误差通常为 $O(\delta_j^2)$，舍入误差则随 δ_j 缩小而增加。代码取 $\delta_j = \epsilon^{1/3}\max(1, |x_j|)$ 是双精度下的实用起点，不适用于所有噪声和量化测量。

如果每次函数求值都来自真实仪器，差分中的测量噪声会被除以 δ_j，小扰动反而可能完全淹没有效信号。第 15 章将讨论重复采样、双侧扰动和量化平台。

7.4 练习与解答

练习：$r_i = ae^{-kt_i} + c - y_i$ 改为 $y_i - ae^{-kt_i} - c$，梯度 $J^T r$ 是否改变？解答：r 与 J 同时变号，梯度不变。

练习：当 $a = 0$ 时，k 的局部可辨识性怎样？解答：雅可比的第二列全零，速率此时无法通过局部观测估计；这是模型信息不足，不是换一个求解器就会消失的问题。

练习：$g = 0$ 是否证明全局最小？解答：不证明，它只说明一阶驻点；非线性目标可能有局部极小、极大或鞍点。

第八章　迭代优化的基本工具

在非线性问题中，“方向大致正确” 和 “步子足够谨慎” 是两项独立任务。先理解这两件事，后面的 GN、LM 和信赖域就不再只是公式名称。

8.1　下降方向和步长

对于可微目标 F，若 $g^T h < 0$，则沿 h 走足够小的正步长会下降。最速下降取 $h = -g$；它在欧氏单位球内给出最大的局部线性下降，但不代表整个优化过程最快。

Armijo 回溯从 $\alpha = 1$ 开始，若

$$F(x + \alpha h) \le F(x) + c\alpha g^T h, \qquad 0 < c < 1,$$

不成立，就缩小 α。这里右侧比当前目标更低，要求下降量与局部预测有合理联系。实现时还必须限制回溯次数或最小步长。

8.2　为什么最速下降会来回摆动

对二次目标 $F(x) = \frac{1}{2} x^T B x$，若 B 的特征值差别很大，等高线狭长，负梯度容易横穿 “山谷”。固定步长稳定性要求 $0 < \alpha < 2/\lambda_{\max}(B)$。沿曲率大的方向稳定的步长，在曲率小的方向就显得很慢。

```
B = np.array([[10., 8.], [8., 10.]])
cost = lambda x: .5*x@B@x
x = np.array([3., 0.]); path = [x.copy()]
for _ in range(35):
    g = B@x; h = -g; alpha = 1.
    while cost(x+alpha*h) > cost(x)+1e-4*alpha*(g@h):
        alpha *= .5
    x = x+alpha*h
    path.append(x.copy())
path = np.array(path)
u, v = np.meshgrid(np.linspace(-1, 3.2, 120), np.linspace(-2, 1.2, 120))
z = .5*(B[0,0]*u*u+2*B[0,1]*u*v+B[1,1]*v*v)
plt.contour(u, v, z, levels=18)
plt.plot(path[:,0], path[:,1], 'o-', markersize=2, label='gradient + backtracking')
plt.plot(0,0,'rx',label='Newton solution')
```

```
plt.xlabel('x1'); plt.ylabel('x2'); plt.legend(); plt.show()
print('gradient method final norm:', np.linalg.norm(x))
```

```
gradient method final norm: 2.0236746917870024e-06
```

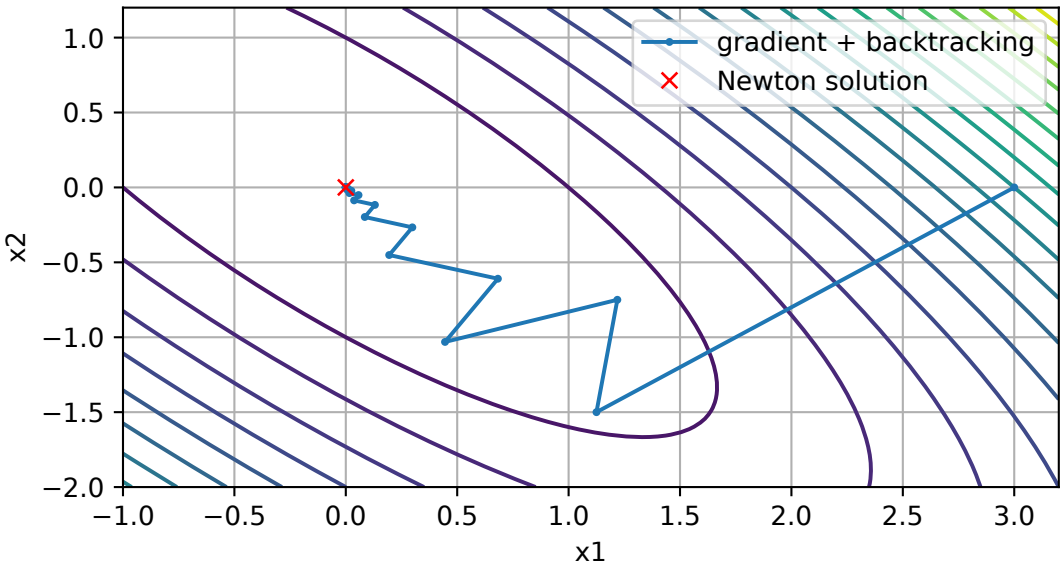


图 8.1: 狭长二次目标上的梯度回溯路径

8.3 牛顿法利用曲率

二阶模型 $F(x+h) \approx F(x) + g^T h + \tfrac{1}{2} h^T H h$ 的驻点满足 $Hh = -g$。若 H 正定，牛顿方向是下降方向；若 H 不定，解这个方程可能向上走。对正定二次目标，精确牛顿法一步到达解；一般非线性目标则不具备这个保证。

下面用一个非凸函数证明“有 Hessian 就一定更稳”是错误的。$F(z) = z^4 - 3z^2$ 在 $z = 0$ 附近曲率为负。

```
z = .3
g = 4*z**3-6*z
H = 12*z**2-6
h = -g/H
F = lambda z: z**4-3*z**2
print('Hessian:', H, 'g*h:', g*h)
print('cost before/after full Newton step:', F(z), F(z+h))
assert g*h > 0
assert F(z+h) > F(z)
```

```
Hessian: -4.92 g*h: 0.5818829268292681
cost before/after full Newton step: -0.2619 -0.005778557493008074
```

8.4 停止规则各自说明什么

梯度小表示一阶变化小；步长小可能是收敛，也可能是尺度差、阻尼过大或数值停滞；目标下降小可能是噪声下限，也可能是走错方向。应记录梯度、步长、目标和预算，而不是只输出一个“成功”。

本书求解器返回参数、目标值、状态和迭代历史。预算耗尽、回溯失败、阻尼导致的停滞分别标识。即使一阶停止条件满足，也仍需检查可辨识性、残差结构和物理合理性。

8.5 练习与解答

练习：如果 h 不是下降方向，回溯一定能找到下降步吗？解答：不能；局部正方向导数意味着充分小的正步长反而上升，应先修正方向。

练习：把 B 换为单位矩阵，解释最速下降为何快很多。解答：等高线变成圆，所有方向尺度相同，合适步长可一步到达中心。

练习：牛顿法的二次收敛是否从任意初值成立？解答：通常只在足够靠近解、Hessian 非奇异且足够光滑等条件下成立，属于局部性质。

第九章　Gauss–Newton 方法

既然第 07 章把非线性残差局部近似为 $r + Jh$，最自然的下一步就是把这个线性最小二乘问题解掉，再更新参数。

9.1　推导一步更新

$$h_{\mathrm{GN}} = \arg\min_{h} \|Jh + r\|^2, \qquad x_{\mathrm{new}} = x + h_{\mathrm{GN}}.$$

满列秩时等价于 $J^T Jh = -J^T r$。实现直接对 J 使用最小二乘，避免形成正规方程。它相当于用 $J^T J$ 近似 Hessian，省略了 $\sum_i r_i \nabla^2 r_i$。

当残差接近零且局部雅可比满秩时，GN 往往很快。大残差、坏初值或强曲率会使局部线性模型失真；此时满步长可能让真实目标大幅上升。

9.2　先写一个未经保护的 GN 循环

Rosenbrock 残差 $r = (10(x_2 - x_1^2),\, 1 - x_1)^T$ 的最小值在 $(1, 1)$。从 $(-1.2, 1)$ 开始，观察第一步目标如何变化。

```
def rfun(x):
    return np.array([10*(x[1]-x[0]**2), 1-x[0]])
def jfun(x):
    return np.array([[-20*x[0], 10.], [-1., 0.]])
x = np.array([-1.2, 1.])
raw = [.5*rfun(x)@rfun(x)]
for _ in range(6):
    h = np.linalg.lstsq(jfun(x), -rfun(x), rcond=None)[0]
    x = x+h
    raw.append(.5*rfun(x)@rfun(x))
print('unprotected full-step costs:', np.array(raw))
assert raw[1] > raw[0]
safe = gauss_newton(rfun, jfun, [-1.2,1.])
print('safeguarded estimate/status:', safe.x, safe.status)
costs = [p['cost'] for p in safe.history]
plt.semilogy(np.maximum(raw, 1e-20), 'o-', label='full GN')
plt.semilogy(np.maximum(costs, 1e-20), '.-', label='GN + backtracking')
plt.xlabel('iteration'); plt.ylabel('cost'); plt.legend(); plt.show()
```

```
assert np.allclose(safe.x, [1,1], atol=1e-6)
```

```
unprotected full-step costs: [  12.1  1171.28    0.      0.      0.      0.      0.  ]
safeguarded estimate/status: [1. 1.] gradient
```

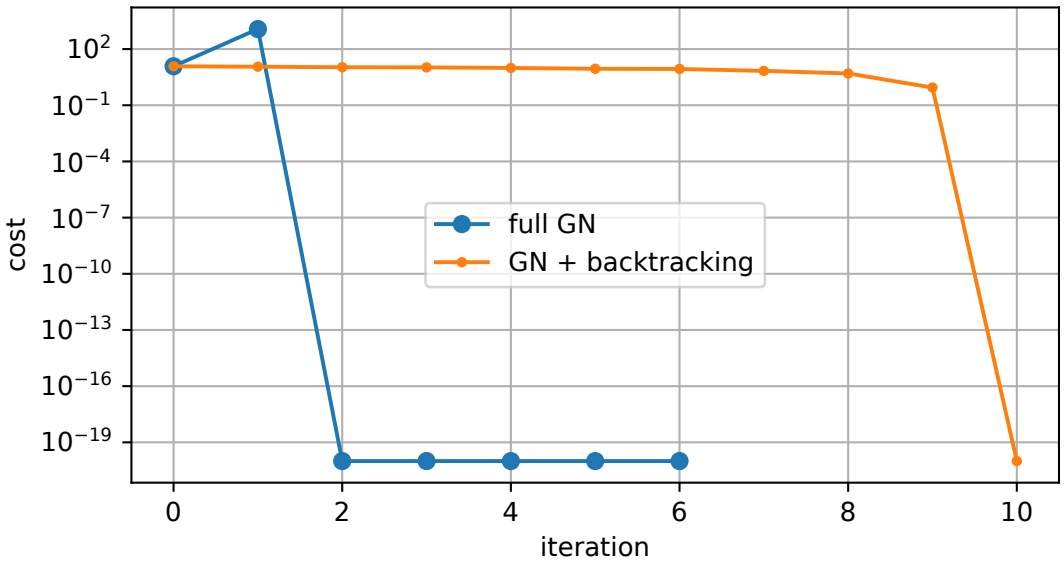


图 9.1: 未经保护与回溯 GN 的目标变化

9.3 给方向加上回溯保护

解出 h 后，检验 $F(x+\alpha h)$，必要时将 α 减半。线性模型没有变，只是承认它在太远处可能不可信。第 08 章的方向与步长分工在这里直接复用。

下面拟合真实目标非零的指数观测，并与 SciPy 比较。此时“拟合成功”不意味着所有残差都变成零，噪声会留下不可消除的误差。

```
rng = np.random.default_rng(109)
t = np.linspace(0, 4, 40)
y = 2.5*np.exp(-.8*t)+.4+rng.normal(0,.035,len(t))
def residual(p):
    return p[0]*np.exp(-p[1]*t)+p[2]-y
def jac(p):
    e = np.exp(-p[1]*t)
    return np.column_stack([e,-p[0]*t*e,np.ones_like(t)])
ours = gauss_newton(residual, jac, [2., .5, .2], gtol=1e-8)
reference = least_squares(residual,[2., .5, .2],jac=jac,gtol=1e-12,ftol=1e-12,xtol=1e-12)
print('ours:', ours.x, ours.status, 'cost:', ours.cost)
print('SciPy:', reference.x, 'cost:', reference.cost)
assert np.allclose(ours.x, reference.x, atol=2e-5)
plt.scatter(t,y,s=12,label='data'); plt.plot(t,residual(ours.x)+y,label='fit')
plt.xlabel('time'); plt.ylabel('response'); plt.legend(); plt.show()
```

```
ours: [2.50686 0.80254 0.4034 ] gradient cost: 0.013858140938455986
SciPy: [2.50686 0.80254 0.4034 ] cost: 0.013858140938455986
```

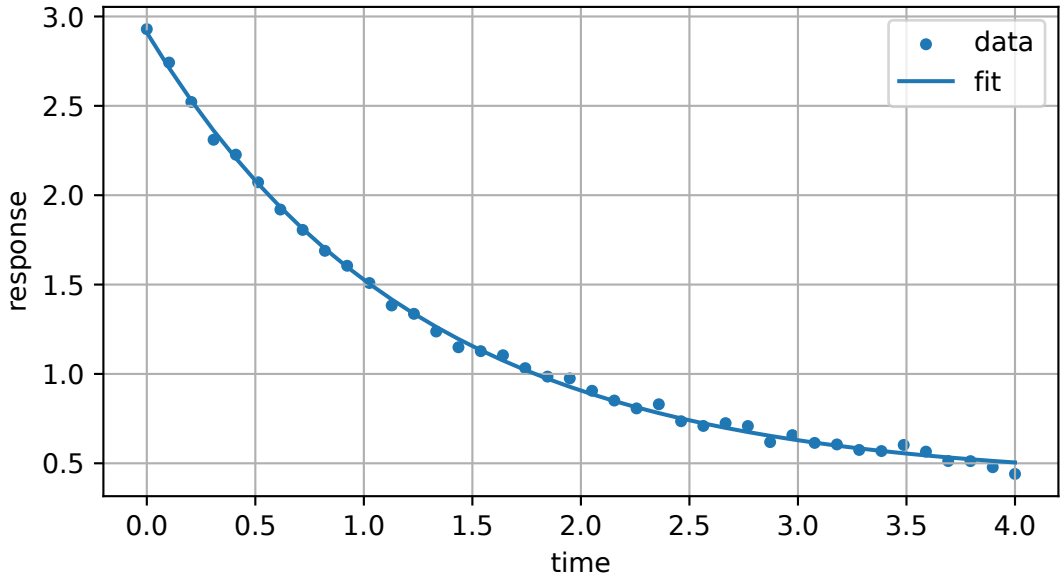


图 9.2: GN 拟合有噪声指数观测

9.4 何时不该继续硬走 GN

若雅可比近秩亏，局部步可能很大；若参数差好几个数量级，某些坐标会主导步长；若噪声使局部线性化不稳定，频繁回溯会很慢。可以改善参数化、缩放、增加信息，或用 LM 和信赖域限制步长。求解器不能修复完全不可辨识的模型。

9.5 练习与解答

练习：在线性残差 $r = Ax - y$ 下，GN 需要几次有效更新？解答：精确运算、满秩且采用完整步时一次到达最小二乘解，因为局部模型就是原问题。

练习：GN 能否用于 $m < n$？解答：线性子问题可求最小范数步，但存在未被观测约束的参数方向，不能声称解唯一；需要先验、约束或更多信息。

练习：上图中未经保护 GN 最后也到达了解，为什么仍需保护？解答：中间的大幅上升在其他模型里可能触发溢出、离开物理范围或进入错误吸引域，最终碰巧成功不是可靠策略。

第十章　Levenberg–Marquardt 方法

GN 有时想迈出过大的步。LM 给局部步增加一个代价，让算法在 “相信线性模型” 和 “谨慎尝试” 之间调整。

10.1　阻尼作用在步长上

$$\min_h \frac{1}{2}\|Jh+r\|^2+\frac{\mu}{2}\|h\|^2, \qquad (J^TJ+\mu I)h=-g.$$

当 μ 大时，$h\approx -g/\mu$，方向接近负梯度；μ 小时接近 GN。与第 06 章固定参数先验不同，这里的惩罚针对当前步 h，通常不作为最终目标的一部分。

数值实现使用增广矩阵 $[J;\sqrt{\mu}I]$ 做最小二乘。既能理解阻尼的意义，也能避免直接形成 J^TJ。

```
def rfun(x):
    return np.array([10*(x[1]-x[0]**2), 1-x[0]])
def jfun(x):
    return np.array([[-20*x[0], 10.], [-1., 0.]])
x = np.array([-1.2,1.]); r = rfun(x); J = jfun(x)
for mu in (.001, 1., 100.):
    augmented = np.vstack([J,np.sqrt(mu)*np.eye(2)])
    h = np.linalg.lstsq(augmented,np.r_[-r,np.zeros(2)],rcond=None)[0]
    predicted = -(J.T@r)@h-.5*(J@h)@(J@h)
    actual = .5*(r@r-rfun(x+h)@rfun(x+h))
    print('mu, step, actual/predicted:',mu,h,actual/predicted)
```

```
mu, step, actual/predicted: 0.001 [ 2.18628 -4.80702] -93.40607323352172
mu, step, actual/predicted: 1.0 [ 0.42134 -0.56555] 0.8597119121338301
mu, step, actual/predicted: 100.0 [0.14139 0.05033] 0.9877615479733826
```

10.2　用增益比检验模型

局部模型预测的下降量为

$$q(0)-q(h)=-g^Th-\frac{1}{2}\|Jh\|^2.$$

真实下降除以预测下降，得到

$$\rho = \frac{F(x) - F(x+h)}{-g^T h - \frac{1}{2}\|Jh\|^2}.$$

预测下降必须为正；否则不能用这个比值声称局部模型可信。若 $\rho > 0$，本书实现接受步；若 $\rho \le 0$ 则保留旧参数并增大阻尼。接受后根据 ρ 降低或调整阻尼。记录拒绝步很重要：它说明算法付出了函数求值，却没有更新解。

当步是阻尼方程的精确解时，分母也等于 $\frac{1}{2}h^T(\mu h - g)$。直接采用前一式更清晰地表明它来自原目标的局部二次模型。

```
ours = levenberg_marquardt(rfun,jfun,[-1.2,1.],max_iter=200)
reference = least_squares(rfun,[-1.2,1.],jac=jfun,method='lm')
print('LM estimate/status:',ours.x,ours.status)
print('SciPy LM:',reference.x)
print('rejected attempts:',sum(not p['accepted'] for p in ours.history))
assert np.allclose(ours.x,reference.x,atol=1e-6)
history = ours.history[1:]
fig,ax = plt.subplots(1,2,figsize=(7,3))
ax[0].semilogy([max(p['cost'],1e-20) for p in ours.history],'.-')
ax[0].set(xlabel='attempt',ylabel='accepted-state cost')
ax[1].semilogy([p['control'] for p in history],'.-')
ax[1].set(xlabel='attempt',ylabel='damping mu')
plt.tight_layout();plt.show()
```

```
LM estimate/status: [1. 1.] gradient
SciPy LM: [1. 1.]
rejected attempts: 2
```

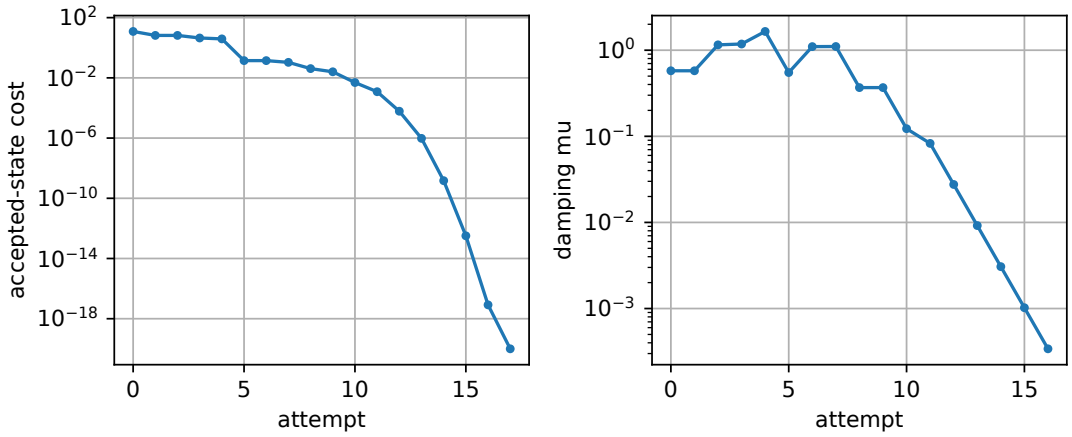


图 10.1: LM 的目标历史与阻尼历史

图中目标保持不变的区段可能对应拒绝步，不能按图上的点数直接计算“有效参数更新次数”。不同库的函数求值次数、迭代次数和拒绝次数定义也未必相同。

10.3 LM 仍然有边界

阻尼改善局部步的稳定性，但不会保证全局最优。参数单位会影响 $\|h\|^2$，所以应先做合理缩放。普通 LM 也不自动满足上下界；有边界时可用第 12 章的有界信赖域求解。真实仪器量化平台上的零雅可比，还需要专门的采样和探测策略。

本书实现中，若大阻尼把步压到很小、梯度却不小，会返回 stalled，而非把这种现象冒充收敛。

10.4 练习与解答

练习：μ 增大一定让真实目标下降吗？解答：不一定；它通常缩小局部步，但仍需真实目标验证。

练习：拒绝一步后是否更新 x？解答：不更新；只改变阻尼并从同一个当前点重新计算候选步。

练习：若预测下降是零，如何处理？解答：不能直接相除；应检查驻点、退化方向或数值问题，并采取明确的停止/拒绝策略。

第十一章　信赖域与狗腿法

LM 通过给步长加惩罚来限制移动。信赖域则直接规定 “这次最多走多远”：在 $\|h\| \le \Delta$ 内最小化局部模型。两者共享同一个核心判断——局部模型能否预测真实下降。

11.1　两个容易计算的候选步

GN 步 h_{GN} 给出线性残差模型的最优解；沿负梯度方向最小化同一个模型，得到 Cauchy 步

$$h_{\mathrm{C}} = -\frac{g^T g}{\|Jg\|^2} g.$$

狗腿法沿折线 $0 \to h_{\mathrm{C}} \to h_{\mathrm{GN}}$ 找信赖域内的点：

1. GN 步在球内，直接使用。
2. Cauchy 步也在球外，沿负梯度截到球面。
3. 否则，取 $h_{\mathrm{C}} + \beta(h_{\mathrm{GN}} - h_{\mathrm{C}})$ 与球面的交点。

第三种情况令 $v = h_{\mathrm{GN}} - h_{\mathrm{C}}$，解

$$\|v\|^2 \beta^2 + 2 h_{\mathrm{C}}^T v\, \beta + \|h_{\mathrm{C}}\|^2 - \Delta^2 = 0,$$

选 $0 \le \beta \le 1$ 的根。这个构造是对信赖域子问题的近似求解，不是精确解的通用公式。

```
x = np.array([-1.2,1.])
r = np.array([10*(x[1]-x[0]**2),1-x[0]])
J = np.array([[-20*x[0],10.],[-1.,0.]])
g = J.T@r
hc = -(g@g)/((J@g)@(J@g))*g
hgn = np.linalg.lstsq(J,-r,rcond=None)[0]
radius = 1.; v = hgn-hc
a,b,c = v@v,2*hc@v,hc@hc-radius**2
beta = (-b+np.sqrt(b*b-4*a*c))/(2*a)
h = hc+beta*v
assert 0 <= beta <= 1
assert np.isclose(np.linalg.norm(h),radius)
theta = np.linspace(0,2*np.pi,200)
plt.plot(radius*np.cos(theta),radius*np.sin(theta),'--',label='trust boundary')
plt.plot([0,hc[0],hgn[0]],[0,hc[1],hgn[1]],'o-',label='dogleg path')
plt.plot(*h,'rx',markersize=10,label='selected step')
```

```
plt.axis('equal');plt.xlabel('h1');plt.ylabel('h2');plt.legend();plt.show()
```

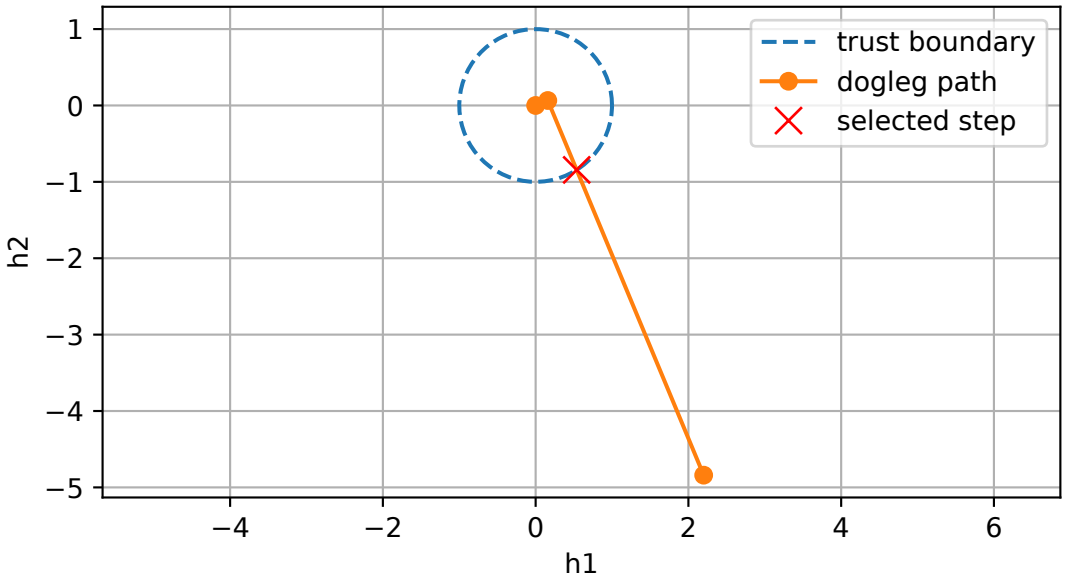


图 11.1: 狗腿折线与信赖域球面交点

11.2 半径如何调整

与 LM 一样，计算真实下降/预测下降的 ρ。本书实现中，$\rho < 0.25$ 时半径缩为四分之一；$\rho > 0.75$ 且步接近边界时扩大半径；$\rho > 0.1$ 才接受更新。这些是具体实现选择，不是唯一允许的阈值。

如果一个很小的步在球内，模型仍很差，继续扩大半径通常没有意义。反之，如果步一直顶到边界而预测很准确，扩大信赖域能加快进展。

```
def fun(x):
    return np.array([10*(x[1]-x[0]**2),1-x[0]])
def jac(x):
    return np.array([[-20*x[0],10.],[-1.,0.]])
results = {
    'GN':gauss_newton(fun,jac,[-1.2,1.]),
    'LM':levenberg_marquardt(fun,jac,[-1.2,1.]),
    'dogleg':dogleg(fun,jac,[-1.2,1.])
}
for name,result in results.items():
    print(name,result.x,result.status,'attempts:',len(result.history)-1)
    plt.semilogy([max(p['cost'],1e-20) for p in result.history],'.-',label=name)
    assert np.allclose(result.x,[1,1],atol=1e-5)
plt.xlabel('attempt (different per-method cost)');plt.ylabel('cost')
plt.legend();plt.show()
```

```
GN [1. 1.] gradient attempts: 10
```

```
LM [1. 1.] gradient attempts: 17
dogleg [1. 1.] gradient attempts: 14
```

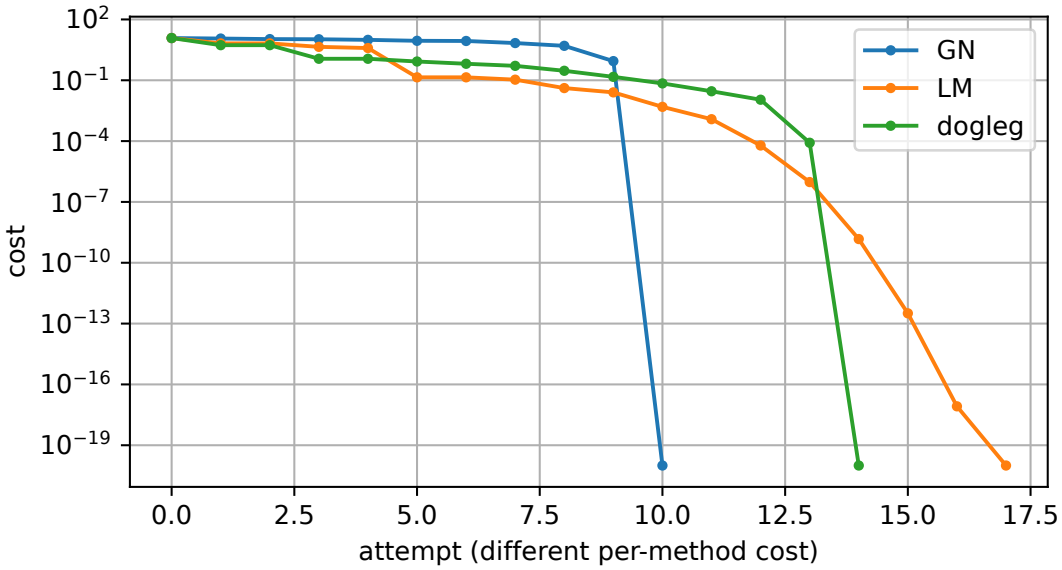


图 11.2：三种局部方法在相同问题上的目标历史

图中的尝试次数不能直接当作运行时间对比：三种方法每步的函数/雅可比求值和线性代数开销并不完全相同。做性能研究应在相同停止精度下统计实际求值与耗时。

原参考书狗腿算法有已记录的符号疑点。这里始终从 $q(0)-q(h)=-g^{T}h-\tfrac{1}{2}\|Jh\|^{2}$ 计算预测下降，避免把针对特殊分支的表达式未经核验搬到一般非零残差问题。

11.3　练习与解答

练习：将初始半径缩小 100 倍，早期步长和迭代次数如何变化？解答：初期更谨慎，通常需要几次半径扩大才进入快速区域；具体总次数仍取决于模型。

练习：SciPy 的 dogbox 是否就是这里的球形狗腿法？解答：不是；dogbox 使用矩形信赖域并处理边界，名称相近但不能按同一算法解释。

第十二章　让算法适应真实数据

真实数据常同时包含不同量纲、边界、异常值和错误初值。把求解器从 GN 换成 LM 通常只解决其中一部分，其他问题需要在建模和验证层处理。

12.1　先把参数尺度写清楚

可令 $x = Sz$，其中 S 是各参数典型尺度组成的对角矩阵。新雅可比为 $J_z = J_x S$。这样一个单位的 z 变化，代表相近的实际重要程度。残差白化和参数缩放分别作用于行与列，含义不同，不能相互替代。

12.2　边界不是裁剪一下就等价

若速率必须非负，应在优化问题中明确 $k \geq 0$。先算无约束步再逐分量裁剪，可能破坏方向与预测下降的关系。SciPy 的 trf 方法支持上下界；其 lm 方法只处理无界问题且仅支持线性损失。边界上的解也不能简单用“所有梯度为零”检查。

12.3　稳健损失降低异常值支配

普通平方损失会放大极端误差。以 soft-L1 为例，令标准化平方残差为 s，使用 $\rho(s) = 2(\sqrt{1+s}-1)$。小残差附近近似平方损失，大残差处增长更慢。尺度参数应与正常噪声量级联系，而不是只为得到喜欢的答案。

```
rng = np.random.default_rng(112)
t = np.linspace(0,4,60); sigma=.04
truth = np.array([2.5,.8,.4])
y = truth[0]*np.exp(-truth[1]*t)+truth[2]+rng.normal(0,sigma,len(t))
y[[5,18,30]] += [1.2,-1.1,1.3]
def residual(p):
    return p[0]*np.exp(-p[1]*t)+p[2]-y
bounds = ([0.,0.,0.],[6.,4.,2.])
ordinary = least_squares(residual,[2.,.5,.2],bounds=bounds,method='trf')
robust = least_squares(residual,[2.,.5,.2],bounds=bounds,
                       method='trf',loss='soft_l1',f_scale=sigma,
                       x_scale=[2.,1.,.5])
print('truth:',truth,'ordinary:',ordinary.x,'robust:',robust.x)
```

```
plt.scatter(t,y,s=12,label='contaminated data')
plt.plot(t,residual(ordinary.x)+y,label='squared loss')
plt.plot(t,residual(robust.x)+y,label='soft-L1')
plt.xlabel('time');plt.ylabel('response');plt.legend();plt.show()
assert np.all(robust.x >= np.array(bounds[0]))
assert np.all(robust.x <= np.array(bounds[1]))
```

```
truth: [2.5 0.8 0.4] ordinary: [2.57621 0.85852 0.44442] robust: [2.48872 0.7942  0.39657]
```

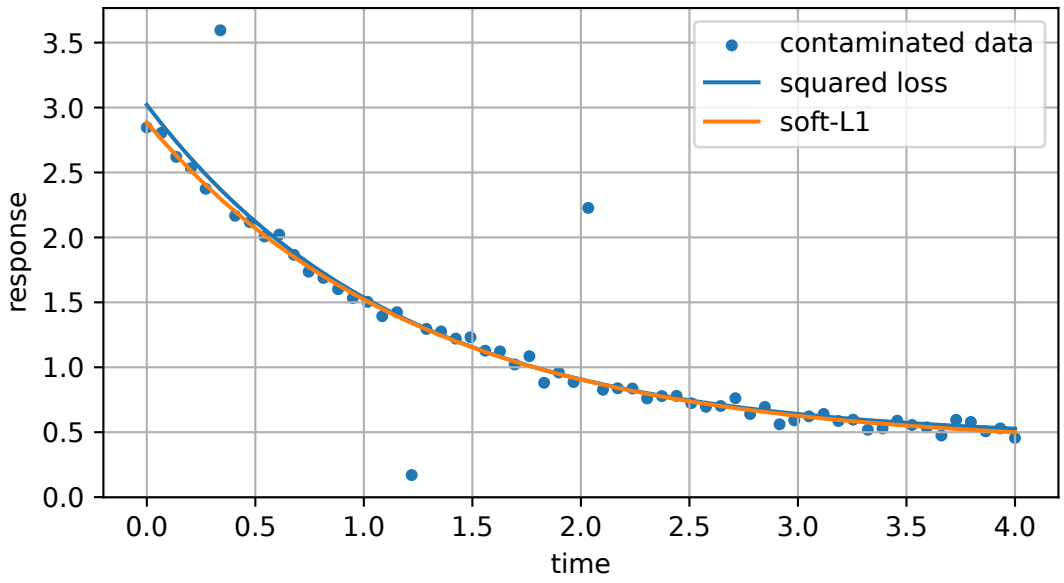


图 12.1: 普通平方与 soft-L1 损失的拟合差别

稳健拟合后仍应调查异常点：设备饱和、通信错误和模型缺失是不同问题。稳健损失不会把一个错误模型自动变成正确模型，也不意味着可以不记录剔除或降权规则。

12.4 一个有顺序的诊断流程

先验证输入有限、单位正确、初值在边界内；再用有限差分核对雅可比；随后检查列尺度与奇异值；最后查看残差随时间、输入和预测值的结构。多初值实验可以揭示局部极小的敏感性，但不能证明已找遍所有解。

为了比较不同算法，应该统一目标定义、数据、初值和停止精度。稳健求解器的 cost 与普通平方损失 cost 定义不同，不能直接把两个数字较小的一方称为“拟合更好”。

```
starts = [[1.,.1,.1],[4.,2.,1.],[2.5,.8,.4]]
for start in starts:
    f = least_squares(residual,start,bounds=bounds,
                      loss='soft_l1',f_scale=sigma,x_scale=[2.,1.,.5])
    raw_rmse = np.sqrt(np.mean(residual(f.x)**2))
    print('start:',start,'x:',f.x,'robust cost:',f.cost,'raw RMSE:',raw_rmse)
```

```
start: [1.0, 0.1, 0.1] x: [2.48872 0.7942  0.39657] robust cost: 0.1796936775748545 raw RMSE:
    0.28659395362999923
start: [4.0, 2.0, 1.0] x: [2.48872 0.7942  0.39657] robust cost: 0.1796936775748545 raw RMSE:
    0.28659395362432843
start: [2.5, 0.8, 0.4] x: [2.48872 0.7942  0.39657] robust cost: 0.1796936775748545 raw RMSE: 0.2865939536366646
```

12.5 练习与解答

练习：把所有残差先除以 σ，soft-L1 的 f_scale 应怎样改？解答：若此前 f_scale 取 σ，标准化后应取 1，以保持相同的残差转折尺度。

练习：采用稳健损失后，能否直接用第 05 章普通最小二乘协方差公式？解答：通常不能；权重随残差变化，且可能有边界，需使用匹配的稳健协方差方法、重采样或明确的近似假设。

练习：目标不再下降时，如何区分已到噪声底与算法失败？解答：结合噪声模型、重复测量、梯度/雅可比、尺度和多初值结果判断；只有一个停止标志并不足够。

第十三章　递推最小二乘

传感器每秒产生一个新样本，反复重新拟合全部历史数据很浪费。递推最小二乘（RLS）把过去的信息压缩到一个参数估计和一个信息矩阵里，每来一条观测就更新。

13.1　从批量目标到递推

设模型 $y_t=\phi_t^{T}x+\varepsilon_t$，考虑

$$S_t(x)=\frac{1}{2}\sum_{i=1}^{t}\lambda^{t-i}(y_i-\phi_i^{T}x)^2+\frac{\lambda^{t}}{2}(x-x_0)^{T}P_0^{-1}(x-x_0).$$

$0<\lambda\le 1$ 是遗忘因子。$\lambda=1$ 保留全部信息，$\lambda<1$ 逐渐降低旧样本权重，也同时衰减初始先验。令 P_t 为加权信息矩阵的逆，矩阵求逆引理给出

$$\begin{aligned}K_t&=\frac{P_{t-1}\phi_t}{\lambda+\phi_t^{T}P_{t-1}\phi_t},\\ \hat{x}_t&=\hat{x}_{t-1}+K_t(y_t-\phi_t^{T}\hat{x}_{t-1}),\\ P_t&=\frac{1}{\lambda}\left(P_{t-1}-K_t\phi_t^{T}P_{t-1}\right).\end{aligned}$$

P_t 在这里是逆信息矩阵；只有结合相应噪声尺度与模型假设，才可解释为参数协方差。初始 P_0 大表示先验弱，但使用有限值仍然对应一个明确的正则项。

```
rng = np.random.default_rng(113)
t = np.linspace(0,5,80); A = np.column_stack([np.ones_like(t),t])
y = A@np.array([1.,2.])+rng.normal(0,.2,len(t))
beta = np.zeros(2); P = 100*np.eye(2); track=[]
for phi, yi in zip(A,y):
    beta,P = recursive_update(beta,P,phi,yi)
    track.append(beta.copy())
batch = np.linalg.solve(A.T@A+.01*np.eye(2),A.T@y)
print('RLS:',beta,'regularized batch:',batch)
assert np.allclose(beta,batch,atol=1e-10)
track=np.array(track)
plt.plot(track[:,0],label='intercept');plt.plot(track[:,1],label='slope')
plt.axhline(1,color='gray',linestyle='--');plt.axhline(2,color='gray',linestyle='--')
plt.xlabel('sample count');plt.ylabel('estimate');plt.legend();plt.show()
```

```
RLS: [1.01505 2.00242] regularized batch: [1.01505 2.00242]
```

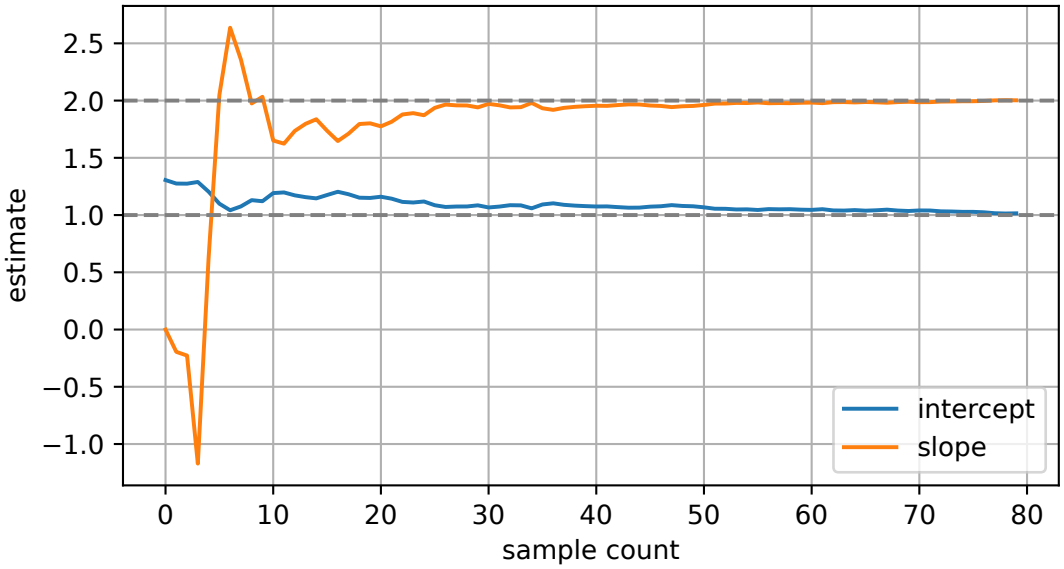


图 13.1: RLS 参数随样本到来的变化

13.2 遗忘使估计能跟踪变化

如果被估参数真的随时间变化，旧数据不一定仍然代表当前系统。常数模型也可以用 RLS 跟踪水平变化；λ 越小，适应越快，但噪声也更明显。有效记忆长度常粗略估为 $1/(1-\lambda)$，这是指数权重和的量级，不是硬截断窗口。

```
signal = np.r_[np.ones(80),np.full(80,2.)]
measurements = signal+rng.normal(0,.15,len(signal))
for lam in (1.,.97):
    b=np.zeros(1);p=100*np.eye(1);estimates=[]
    for yi in measurements:
        b,p=recursive_update(b,p,[1.],yi,forgetting=lam)
        estimates.append(b[0])
    plt.plot(estimates,label=f'lambda={lam}')
plt.plot(signal,'k--',label='changing truth')
plt.xlabel('time index');plt.ylabel('level');plt.legend();plt.show()
```

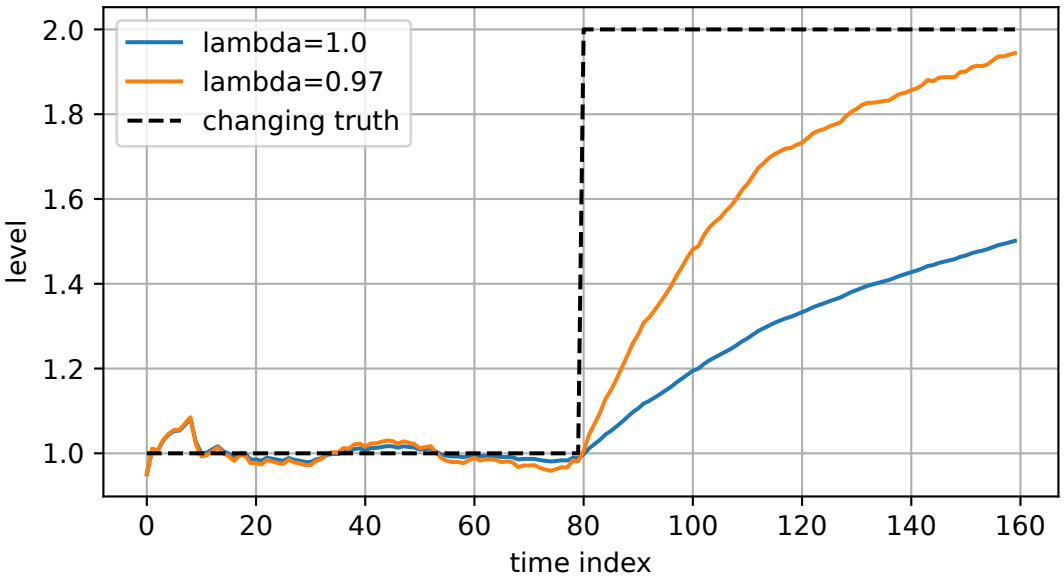


图 13.2: 遗忘因子对变化水平的跟踪

13.3 与卡尔曼滤波的联系和差别

常参数、线性观测、适当先验和噪声假设下，RLS 可视为静态参数卡尔曼估计的一种形式。显式参数动力学和过程噪声可以表达参数如何变化；遗忘因子则用简单的统一折扣近似处理旧信息。两者有联系，但不能不说明模型就把所有 RLS 变体都叫同一个卡尔曼滤波器。

长时间运行可能积累对称性和正定性误差。直接相减的协方差公式在很大的初始 P 下甚至一步就可能发生消去误差。教学实现采用等价的 Joseph 形式：令 $M = I - K_t\phi_t^T$，计算 $P_t = M(P_{t-1}/\lambda)M^T + K_tK_t^T$，再对称化并检查输入正定性。这个写法对应本章单位观测噪声的信息尺度，代价是更多矩阵乘法；对高精度长期系统可研究平方根或 QR 形式，不能把本章的小型实现直接视为工业保证。

13.4 练习与解答

练习：为什么上面的批量对照多了 $0.01I$？解答：$P_0 = 100I$，其逆为 $0.01I$，初始零均值先验仍然存在。若忽略它，RLS 与批量比较就不是同一个目标。

练习：$\lambda = .97$ 时，100 步前的权重是多少？解答：约为 $.97^{100} \approx 0.0476$，不是零。

练习：参数不变且观测噪声独立时，是否应该总用很小的 λ？解答：不应该；遗忘会舍弃有效信息，增加估计方差，只有模型确实需要适应变化时才有相应理由。

第十四章　综合案例：曲线拟合与标定

现在把建模、初始化、加权求解、残差诊断和不确定性放进同一个流程。任务是从一个衰减响应估计幅值 a、衰减速率 k 和背景值 c。

14.1　在拟合前固定实验规则

数据来自仓库中的 decay-calibration.csv，明确为合成数据。时间单位为秒，响应使用任意一致单位，每行给出已知噪声标准差。固定留出每五行中的第一行，其余用于拟合；留出数据不参与初值和参数选择。

模型为 $f(t) = ae^{-kt} + c$，采用白化残差 $(f - y)/\sigma$。a、k 和 c 在本例均限制为非负；这是本合成响应对背景值的假设，其他仪器的背景值可能允许为负。初值不读取仿真真参数，而是在一组候选 k 上，对 a, c 做线性最小二乘，并选择训练误差最小的一组。

```
data=np.genfromtxt(root/'data/decay-calibration.csv',delimiter=',',names=True)
t,y,sigma=data['time_s'],data['response'],data['noise_sigma']
train=np.arange(len(t))%5!=0
tt,yy,ss=t[train],y[train],sigma[train]
candidates=[]
for k in np.linspace(.1,1.5,50):
    A=np.column_stack([np.exp(-k*tt),np.ones_like(tt)])
    ac=np.linalg.lstsq(A/ss[:,None],yy/ss,rcond=None)[0]
    if ac[0]>=0:
        cost=np.sum(((A@ac-yy)/ss)**2)
        candidates.append((cost,np.array([ac[0],k,ac[1]])))
initial=min(candidates,key=lambda z:z[0])[1]
initial[2]=max(0.,initial[2])
def model(p,t):
    return p[0]*np.exp(-p[1]*t)+p[2]
def residual(p):
    return (model(p,tt)-yy)/ss
def jac(p):
    e=np.exp(-p[1]*tt)
    return np.column_stack([e,-p[0]*tt*e,np.ones_like(tt)])/ss[:,None]
fit=least_squares(residual,initial,jac=jac,bounds=(0,np.inf),x_scale=[3,1,.3])
print('data-derived initial:',initial,'fit:',fit.x,'status:',fit.message)
print('training normalized RMSE:',np.sqrt(np.mean(residual(fit.x)**2)))
heldout=(model(fit.x,t[~train])-y[~train])/sigma[~train]
print('held-out normalized RMSE:',np.sqrt(np.mean(heldout**2)))
```

```
assert fit.success and np.all(np.isfinite(fit.x))
```

```
data-derived initial: [2.78596 0.67143 0.2823 ] fit: [2.79075 0.66143 0.26947] status: `ftol` termination
    condition is satisfied.
training normalized RMSE: 0.8496890299942883
held-out normalized RMSE: 1.07904696484996
```

14.2 可辨识性和局部不确定性

白化雅可比的奇异值跨度可以提示局部弱方向。这里噪声尺度是已知的，先在雅可比数值满列秩、没有活动边界时，用局部线性化估计 $C_x \approx (J^T J)^{-1}$，不再额外乘训练残差方差。不满足这些条件时，本例明确省略参数区间和均值置信带；伪逆不能把不可辨识方向变成已知的零方差。若约束处于活动边界或模型明显非线性，这个正态近似会失真。

对新时间 t，模型均值的一阶方差为 $j(t)C_x j(t)^T$。这描述平均响应的不确定性；未来一次带噪声观测的预测方差还要加观测噪声方差。两种区间不可混为一谈。

```
from lsq import linearized_covariance
J=jac(fit.x)
s=np.linalg.svd(J,compute_uv=False)
print('Jacobian singular values:',s)
try:
    cov=linearized_covariance(J,fit.active_mask)
    print('approximate parameter standard errors:',np.sqrt(np.diag(cov)))
except ValueError as error:
    cov=None
    print('uncertainty omitted:',error)
grid=np.linspace(0,5,160);e=np.exp(-fit.x[1]*grid)
Jmean=np.column_stack([e,-fit.x[0]*grid*e,np.ones_like(grid)])
prediction=model(fit.x,grid)
fig,ax=plt.subplots(1,2,figsize=(7,3))
ax[0].scatter(tt,yy,s=10,label='train')
ax[0].scatter(t[~train],y[~train],marker='x',label='held-out')
ax[0].plot(grid,prediction)
if cov is not None:
    se=np.sqrt(np.maximum(0,np.einsum('ij,jk,ik->i',Jmean,cov,Jmean)))
    ax[0].fill_between(grid,prediction-1.96*se,prediction+1.96*se,
                       alpha=.25,label='mean 95% approx.')
ax[0].set(xlabel='time (s)',ylabel='response');ax[0].legend(fontsize=7)
ax[1].scatter(tt,residual(fit.x),s=12)
ax[1].axhline(0,color='gray');ax[1].set(xlabel='time (s)',ylabel='normalized residual')
plt.tight_layout();plt.show()
```

```
Jacobian singular values: [243.55618  57.36661  29.09522]
approximate parameter standard errors: [0.02382 0.01723 0.02525]
```

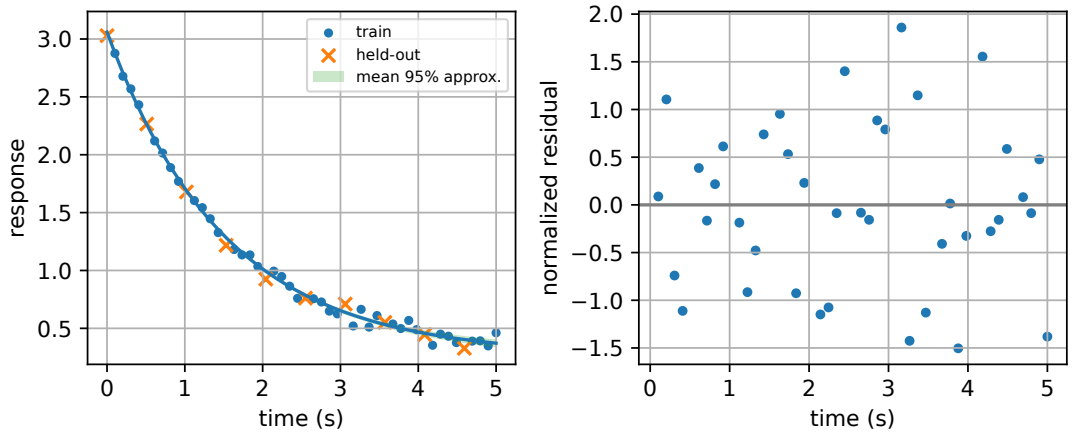


图 14.1: 完整标定结果、均值置信带与标准化残差

本例知道真模型，仍然不能要求每次仿真都把参数估到真值或让 95% 区间覆盖真值。概率陈述描述重复实验，不是一次实验的确定承诺。

14.3 将结果写成可以复查的结论

报告应包含模型、参数单位、数据筛选和划分、噪声/权重来源、求解方法、初值和边界、停止原因、训练/留出残差及不确定性假设。仅报告三个参数和一个“成功”不够。

14.4 练习与解答

练习：只保留很早的时间点，a 和 c 是否容易分开？解答：不容易；当 kt 很小时，$ae^{-kt}+c \approx (a+c)-akt$，数据主要约束 $a+c$ 与 ak 两个组合。增加较晚的采样可提供背景信息。

练习：用同一批留出数据反复选择模型阶数后，还能把它称为最终独立评估吗？解答：不能；它已经参与模型选择，需要新的测试数据或嵌套验证。

练习：若残差随时间有系统曲率，扩大参数误差条能否解决？解答：不能；应先怀疑模型缺失、时间响应或噪声假设，协方差公式不能修复系统偏差。

第十五章　综合案例：PWM→XYZ 校正

已知目标三刺激值，希望调整三通道 PWM 让实测 XYZ 逼近目标。这是一个三输入、三输出、可能含量化和测量噪声的逆问题。本章展示可复现实验框架，不将合成曲线当作真实屏幕的测量结论。

15.1　明确仿真假设

PWM 取 12 位整数 0—4095。低于 96 视为关闭；96 以上的各通道响应连续、分段线性且单调。一个固定的正混色矩阵把通道发光量映射成归一化 XYZ。单次测量加入标准差 2×10^{-4} 的独立高斯噪声。真实设备的通道耦合、热漂移或非单调性需要重新测量，不能直接照搬此模型。

目标函数仍可写为 $\frac{1}{2}\|M(p)-X_{\text{target}}\|^2$。若 XYZ 各分量噪声不同，应白化；本例三者采用同一噪声尺度。

```
mix=np.array([[.52,.29,.18],[.23,.70,.07],[.03,.12,.94]])
knots=np.array([0,96,512,2048,4095.])
levels=np.array([[0,0,.045,.40,1.],
                 [0,0,.055,.44,1.],
                 [0,0,.035,.37,1.]])
def simulator(p):
    p=np.clip(np.rint(p),0,4095)
    light=np.array([np.interp(p[i],knots,levels[i]) for i in range(3)])
    return mix@light
rng=np.random.default_rng(115)
def measure(p,repeats=8):
    return simulator(p)+rng.normal(0,2e-4,(repeats,3)).mean(axis=0)
def measured_jacobian(p,delta=64):
    J=np.zeros((3,3))
    for k in range(3):
        plus=p.copy();minus=p.copy()
        plus[k]=min(4095,p[k]+delta)
        minus[k]=max(96,p[k]-delta)
        span=plus[k]-minus[k]
        J[:,k]=(measure(plus)-measure(minus))/span
    return J
target=simulator([820,960,740])
start=np.array([1100.,900.,1000.])
print('target XYZ:',target,'initial error:',np.linalg.norm(simulator(start)-target))
grid=np.linspace(0,4095,200)
```

```
for k in range(3):
    plt.plot(grid,np.interp(grid,knots,levels[k]),label='RGB'[k])
plt.xlabel('12-bit PWM');plt.ylabel('channel response');plt.legend();plt.show()
```

```
target XYZ: [0.12418 0.14976 0.1032 ] initial error: 0.06697135746402544
```

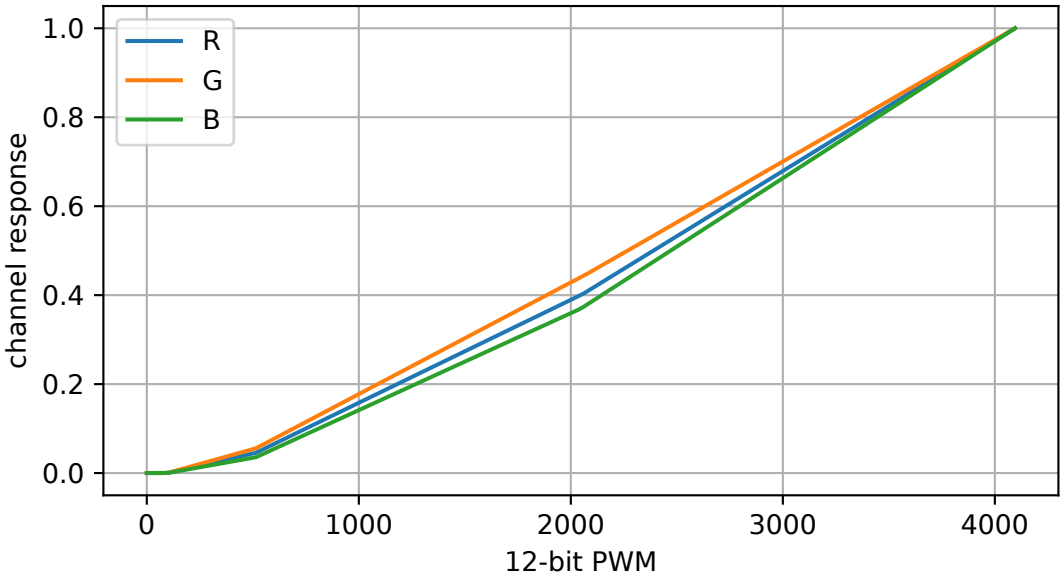


图 15.1: PWM 通道的合成分段单调响应

15.2 用实际扰动跨度估计双侧雅可比

每次只扰动一个通道，另外两通道固定。边界附近的实际差分跨度可能小于 2δ，分母必须使用实际跨度。扰动需要大于量化与噪声造成的不可分辨区，又不能大到跨越太多不同斜率段。重复测量只能在噪声足够独立时降低均值方差；系统漂移不会自动平均掉。

下面把 PWM 参数归一化为 $u = p/4095$，避免导数尺度过小。先用阻尼局部最小二乘求步，再限制单次 PWM 改变量，量化后测量真实下降；失败则增大阻尼。始终保留误差最小的已测点。

```
def calibrate(target,start,max_iter=45,tolerance=1.2e-3):
    target=np.asarray(target,dtype=float)
    start=np.asarray(start,dtype=float)
    if (target.shape!=(3,) or start.shape!=(3,) or
            not np.all(np.isfinite(target)) or not np.all(np.isfinite(start))):
        raise ValueError('target and start must be finite three-vectors')
    if (not isinstance(max_iter,(int,np.integer)) or max_iter<1 or
            np.ndim(tolerance)!=0 or not np.isfinite(tolerance) or tolerance<=0):
        raise ValueError('positive finite tolerance and integer budget required')
    p=np.clip(np.rint(start),96,4095).astype(float)
    lower=simulator([96]*3);upper=simulator([4095]*3)
    box_distance=np.linalg.norm(np.clip(target,lower,upper)-target)
    if box_distance>tolerance:
```

```
        return p,'unreachable_box',[]
    current=measure(p);error=np.linalg.norm(current-target)
    best=p.copy();best_error=error
    mu=1e-4;history=[error];rejects=0
    for _ in range(max_iter):
        if error <= tolerance:
            return p,'target_reached',history
        J=measured_jacobian(p)*4095
        residual=current-target
        du=np.linalg.lstsq(np.vstack([J,np.sqrt(mu)*np.eye(3)]),
                           np.r_[-residual,np.zeros(3)],rcond=None)[0]
        step=4095*du
        step*=min(1.,128/max(np.max(np.abs(step)),1e-12))
        candidate=np.clip(np.rint(p+step),96,4095)
        if np.array_equal(candidate,p):
            return best,'quantized_stall',history
        value=measure(candidate);candidate_error=np.linalg.norm(value-target)
        if candidate_error < error:
            p,current,error=candidate,value,candidate_error
            rejects=0;mu=max(mu/3,1e-9)
            if error<best_error:
                best,best_error=p.copy(),error
        else:
            rejects+=1;mu*=4
        history.append(error)
        if rejects>=8:
            return best,'no_measured_progress',history
    if error<=tolerance:
        return p,'target_reached',history
    return best,'budget_exhausted',history
p,status,history=calibrate(target,start)
print('PWM:',p,'status:',status,'noise-free error:',np.linalg.norm(simulator(p)-target))
assert np.all((p>=96)&(p<=4095))
assert np.linalg.norm(simulator(p)-target)<.003
plt.semilogy(history,'o-');plt.axhline(1.2e-3,color='gray',linestyle='--')
plt.xlabel('attempt');plt.ylabel('measured XYZ error');plt.show()
_,unreachable,_=calibrate(np.array([2.,2.,2.]),start)
print('unreachable target:',unreachable)
assert unreachable != 'target_reached'
```

```
PWM: [821. 959. 739.] status: target_reached noise-free error: 0.00026654803100767464
unreachable target: unreachable_box
```

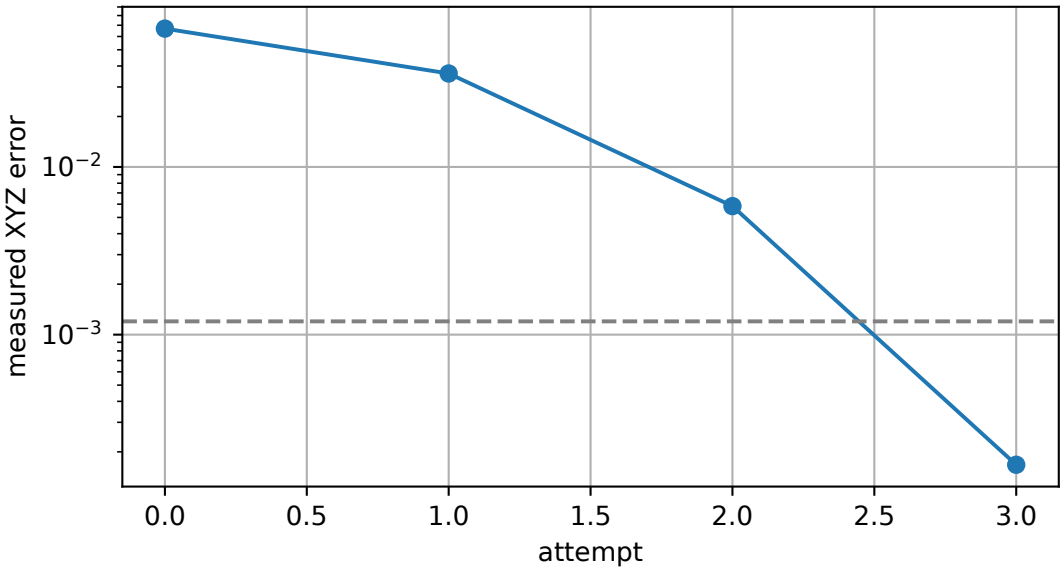


图 15.2: 带噪声 PWM 校正的实测误差历史

15.3 低灰平台和不可达目标

若扰动后量化值完全没变，数值雅可比可能为零。这时梯度小并不说明 XYZ 已经达到目标，应增大有效扰动、跨平台探测或报告无法分辨。本例从 96 以上开始，边界采用实际可用跨度；它并没有解决所有真实低灰平台问题。

输入先验证为有限的三维向量，并要求正的容差与迭代预算。目标到 XYZ 包围盒的距离超过容差时才提前判为不可达；这样容差同样适用于上下两侧。包围盒检查只能排除一部分不可达目标。即使目标的 X、Y、Z 分别都不超限，三者组合仍可能在设备色域外。应在预算、边界和重复测量下报告剩余误差，而不是永远迭代或伪称成功。

本例以一次平均测量的误差选择最好点，会有噪声选择偏差。真实验收应对最终点重新独立多次测量，按 XYZ 或色度/亮度各自容差确认；也要记录热稳定时间、测量顺序和环境条件。

15.4 练习与解答

练习：为什么不直接对 XYZ 误差逐分量分别调 R、G、B？解答：混色矩阵使每个通道同时影响多个 XYZ 分量，需要考虑耦合。

练习：把差分扰动从 64 改为 1，会怎样？解答：有效信号变小，导数中的测量噪声被放大，量化平台也更明显；具体效果可通过重复运行观察。

练习：如果增加 B 时实测 Y 下降，本模型是否足以解释？解答：不能；本模型混色系数和通道斜率均非负。真实现象需要排查耦合、测量时序、限流、热状态和图样条件，先验证模型再选择求解器。

附录 A　必要数学基础

本附录按“在正文哪里会用到”组织。第一次阅读不必记住所有名词，能够识别维度、检查方向和解释条件即可。

A.1　向量、内积与范数

向量 $x \in \mathbb{R}^n$ 有 n 个分量，内积 $x^T y = \sum_j x_j y_j$ 衡量方向关系。$\|x\|_2 = \sqrt{x^T x}$ 是欧氏长度，$\|x\|_\infty = \max_j |x_j|$ 用于逐分量停止判断。$x^T y = 0$ 表示正交，只有两个向量都非零时，才可以进一步定义几何夹角。

矩阵 $A \in \mathbb{R}^{m\times n}$ 把 n 维参数映射到 m 维观测；A^T 交换行列。AB 可乘要求 A 列数等于 B 行数。编程中的一维数组没有显式行列方向，应根据运算结果的 shape 检查含义。

A.2　导数、梯度与二阶展开

标量目标 F 的梯度是所有一阶偏导组成的列向量。向量残差 r 的导数是雅可比 J。链式法则给出 $\nabla(\frac{1}{2}r^T r) = J^T r$。足够光滑时

$$F(x+h) = F(x) + g^T h + \frac{1}{2}h^T H h + O(\|h\|^3).$$

三阶余项写法需要相应光滑性条件。若只假设二阶连续可微，更一般地写余项 $o(\|h\|^2)$。

A.3　正定性、特征值与奇异值

实对称矩阵 B 正定，指所有非零 x 满足 $x^T B x > 0$，等价于所有特征值为正。此时二次目标有唯一极小点，可作 Cholesky 分解。半正定允许零方向，不保证唯一性。$J^T J$ 总半正定，J 满列秩时才正定。

一般矩阵的奇异值非负，平方等于 $A^T A$ 的特征值。最大/最小非零奇异值之比描述可辨子空间上的敏感性；秩亏时通常说整个逆问题的条件数无穷大，不能把有限非零比值当作全参数可逆性。

```
A=np.array([[1.,0.],[1.,1.],[1.,2.]])
singular=np.linalg.svd(A,compute_uv=False)
eigen=np.linalg.eigvalsh(A.T@A)
```

```
print('singular values squared:',singular**2)
print('eigenvalues (ascending):',eigen)
assert np.allclose(np.sort(singular**2),eigen)
```

```
singular values squared: [7.16228 0.83772]
eigenvalues (ascending): [0.83772 7.16228]
```

A.4　概率与协方差

期望描述重复实验的平均位置，协方差 $C=\mathbb{E}[(x-\mathbb{E}x)(x-\mathbb{E}x)^T]$ 描述波动及相关方向。线性变换 $z=Bx$ 后，$\mathrm{Cov}(z)=BCB^T$。不相关仅指协方差为零，一般不等于独立；联合高斯等附加条件下才有更强结论。

A.5　练习与解答

练习：A 为 20×3，r 为 20 维，J^Tr 是什么维度？解答：3 维，与参数维度一致。

练习：J 秩亏时，加 μI 的 $J^TJ+\mu I$ 是否正定？解答：$\mu>0$ 时是；对非零 h，$h^T(J^TJ+\mu I)h=\|Jh\|^2+\mu\|h\|^2>0$。这改善局部步求解，但没有让原数据突然能够辨识所有参数。

附录 B　符号、公式与算法选择

B.1　符号速查

符号	含义	维度
x, h	参数、参数更新	n
y, r	观测、模型减观测的残差	m
A, J	线性设计矩阵、非线性残差雅可比	$m \times n$
F, g, H	半平方和、梯度、精确 Hessian	标量、n、$n \times n$
C	观测噪声协方差	$m \times m$
μ, Δ	LM 阻尼、信赖域半径	标量
λ, D	正则强度、惩罚算子	标量、按问题指定

RLS 章的 λ 表示遗忘因子，语境与第 06 章的正则强度不同。符号相同不代表量的含义相同。

B.2　选择方法前先选择目标

情况	起点	仍需检查
线性、规模适中	QR 或 SVD/lstsq	秩、条件数、参数单位
已知观测协方差	白化后 QR/SVD	协方差来源、正定性
弱可辨识、明确先验	增广正则最小二乘	惩罚尺度、验证集
光滑非线性、初值合理	GN 加回溯	雅可比、模型适用范围
无界光滑非线性	LM	缩放、阻尼历史、局部性
步长需明确限制	信赖域/狗腿	预测下降与真实下降
有上下界或稳健损失	SciPy trf	可行初值、损失尺度
线性数据连续到来	RLS	参数变化、遗忘与数值漂移
量化、平台、含测量噪声	测量设计加受限局部校正	有效扰动、重复测量、可达性

B.3 六项交付检查

明确模型和单位；核对残差与雅可比；查看秩和尺度；记录停止原因；检查未参与拟合的数据；把概率近似和工程容差分别说明。依次做这些检查通常比不断更换求解器更有效。

B.4 公式速查

线性驻点：$A^T(Ax-y)=0$。非线性梯度：$g=J^Tr$。GN：$\min_h \|Jh+r\|^2$。LM：$\min_h \|Jh+r\|^2+\mu\|h\|^2$。预测下降：$-g^Th-\frac{1}{2}\|Jh\|^2$。参数缩放 $x=Sz$ 后：$J_z=J_xS$。

B.5 一段可以运行的初步诊断

```
def diagnose_matrix(A):
    s=np.linalg.svd(A,compute_uv=False)
    rank=np.linalg.matrix_rank(A)
    full_column_rank=rank==A.shape[1]
    condition=(s[0]/s[-1]) if full_column_rank else np.inf
    return {'shape':A.shape,'rank':rank,'full_column_rank':full_column_rank,
            'condition':condition}
print(diagnose_matrix(np.column_stack([np.ones(5),np.arange(5.)])))
print(diagnose_matrix(np.ones((5,2))))
assert not diagnose_matrix(np.ones((5,2)))['full_column_rank']
```

```
{'shape': (5, 2), 'rank': np.int64(2), 'full_column_rank': np.True_, 'condition': np.float64(4.738720018687268)}
{'shape': (5, 2), 'rank': np.int64(1), 'full_column_rank': np.False_, 'condition': inf}
```

练习：若矩阵诊断很好，是否说明残差没有模型偏差？解答：不说明；矩阵诊断只描述当前线性信息结构，残差诊断检验的是数据与模型之间的关系。

附录 C　进阶方法的入口

前面的方法已经足以处理许多中小规模问题。本附录再给出两个可运行的进阶构造，并解释原书中的混合与割线方法要解决什么问题。

C.1　可分最小二乘：先消去线性参数

若模型为 $\Phi(\theta)c$，对固定的非线性参数 θ，c 可以线性求解。定义

$$c^*(\theta)=\arg\min_c \|\Phi(\theta)c-y\|^2,\qquad \psi(\theta)=\frac{1}{2}\|\Phi(\theta)c^*(\theta)-y\|^2.$$

外层只优化 θ，可能显著减少非线性维度。对 $ae^{-kt}+c$，只需外层优化 k，内层求 a,c。若内层矩阵秩随 θ 改变，投影目标的光滑性与导数要特别谨慎。

```
rng=np.random.default_rng(203)
t=np.linspace(0,4,40)
y=2.5*np.exp(-.8*t)+.4+rng.normal(0,.025,len(t))
def linear_part(k):
    return np.column_stack([np.exp(-k*t),np.ones_like(t)])
def projected_residual(theta):
    A=linear_part(theta[0])
    ac=np.linalg.lstsq(A,y,rcond=None)[0]
    return A@ac-y
outer=least_squares(projected_residual,[.5],bounds=([.01],[3.]))
ac=np.linalg.lstsq(linear_part(outer.x[0]),y,rcond=None)[0]
print('variable projection [a,k,c]:',np.r_[ac[0],outer.x[0],ac[1]])
assert outer.success
grid=np.linspace(.1,1.5,100)
plt.plot(grid,[.5*np.sum(projected_residual([k])**2) for k in grid])
plt.axvline(outer.x[0],color='C1')
plt.xlabel('nonlinear rate k');plt.ylabel('profiled cost');plt.show()
```

```
variable projection [a,k,c]: [2.49392 0.81201 0.4154 ]
```

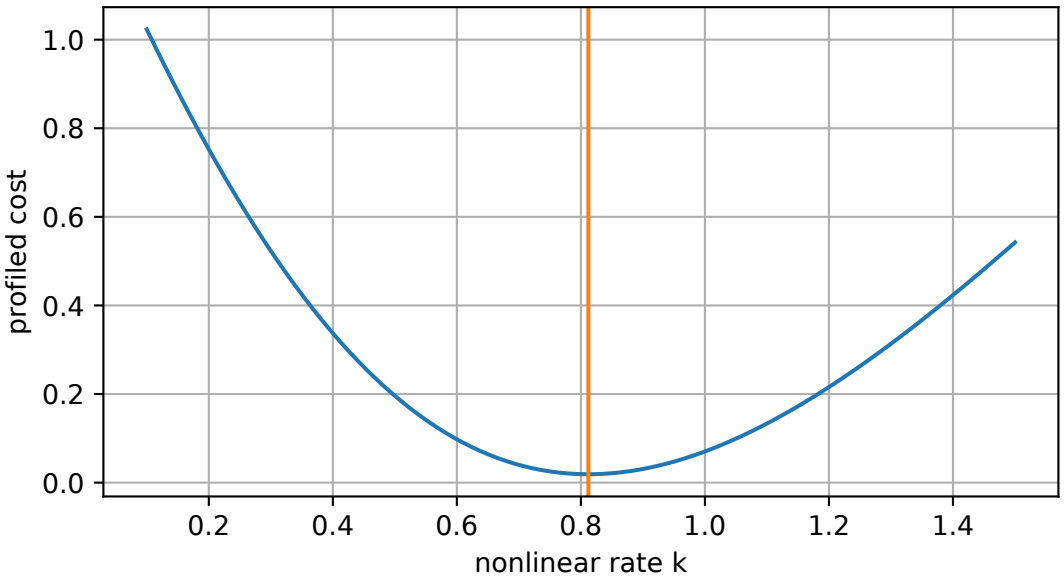


图 C.1: 消去线性参数后的单参数目标

上例让 SciPy 对投影残差做数值微分。它没有把内部最优系数当作与 k 无关的常数；若自行推导外层雅可比，需要包含其随 k 变化的影响，或使用专门的变量投影导数。

C.2 割线更新：用一次新观测修正雅可比

假设走了 $s = x_{k+1} - x_k$，残差变化为 $y_r = r(x_{k+1}) - r(x_k)$。希望近似雅可比 B 满足割线条件 $B_{\text{new}} s = y_r$。一个秩一更新为

$$B_{\text{new}} = B + \frac{(y_r - Bs)s^T}{s^T s}.$$

它在 Frobenius 范数意义下给出满足该割线条件的最小改动。小步、噪声、过期信息都会使更新不可靠，需要重算、保护与接受规则。

```
B=np.array([[1.,.2],[.1,2.],[.4,.3]])
s=np.array([.3,-.2]);delta_r=np.array([.5,-.1,.2])
updated=B+np.outer(delta_r-B@s,s)/(s@s)
print('secant error:',np.linalg.norm(updated@s-delta_r))
assert np.allclose(updated@s,delta_r)
```

```
secant error: 5.551115123125783e-17
```

C.3 混合 LM 与拟牛顿的动机

GN 省略 $\sum_i r_i \nabla^2 r_i$。残差较大且这个项不可忽略时，可尝试用拟牛顿信息近似缺失曲率，并与阻尼/信赖域的保护结合。原书的混合算法还包含具体切换、更新与重置规则；不是给 $J^T J$ 随意

加一个矩阵就完成了混合方法。

本附录实现了可分问题与割线条件实验；原书混合 LM、割线 LM 和割线狗腿的完整工程求解器不在初版实现范围。进一步阅读原书 3.4—3.6 节时，重点核对矩阵近似对象、割线条件、正定性与失败后的恢复策略。

C.4 练习与解答

练习：若 $s = 0$，上面的割线更新可直接使用吗？解答：不能，分母为零；若 s 极小，还会放大噪声，应跳过更新或重建雅可比。

练习：可分最小二乘是否总比联合优化更好？解答：不一定。内层约束、秩变化、求解成本和导数结构都会影响表现，应比较同一个目标和停止精度下的实际结果。

附录 D　参考来源、对应关系与勘误

本教材借鉴可执行教材的组织方式，并按自学顺序重构最小二乘内容。它与保留原章节的中文译本用途不同：教材先补线性基础，再用实验解释非线性算法；译本保留原著结构，供逐节核对。

D.1　主要来源

1. Roger R. Labbe Jr., *Kalman and Bayesian Filters in Python*。项目地址：Labbe 项目 。参考其将文字、公式、代码、图形和练习解答放在 Notebook 中的教学方式。原项目正文采用 CC BY 4.0、代码采用 MIT 许可；本教材没有复制其章节正文或示例代码。
2. K. Madsen, H. B. Nielsen, O. Tingleff, *Methods for Non-Linear Least Squares Problems*, 2nd ed., April 2004。官方入口：DTU 官方入口 。
3. NumPy 官方文档：numpy.linalg.lstsq，说明最小范数解、秩、奇异值及残差摘要的返回条件：NumPy 官方说明 。
4. SciPy 官方文档：scipy.optimize.least_squares，说明 trf/lm/dogbox、边界、损失与尺度：SciPy 官方说明 。

依赖 API 随版本可能变化，复现优先使用仓库记录的验证版本，库的当前文档用于进一步查询。

D.2　与 2004 年原书的对应

本教材	原书位置	阅读重点
01—06 章	作为新增基础；附录 B相关	线性建模、稳定解法和噪声
07 章	第 1 章与第 3 章开头	残差、梯度、Hessian、线性化
08 章	第 2 章 2.1—2.3 节	下降、牛顿、线搜索、局部收敛
09 章	第 3.1 节	GN 及其适用条件
10 章	第 3.2 节	LM 阻尼与增益比
11 章	第 2.4、3.3 节	信赖域、Cauchy 与狗腿
附录 C	第 3.4—3.7 节	混合、割线和可分问题
附录 A、第 04 章	附录 A/B	正定性、SVD 和最小范数

本书统一 $r = f - y$，原书部分使用 $y - f$；引用时先检查符号约定。原书页码与中文重排页码

不同，精确定位以参考工程中的检查记录和页源映射为准。

D.3 如何处理原书已记录的疑点

归档译本采取忠实保留原文并单列疑点的方式。教材采用明确推导与维度检查，不把疑似误植当作算法规范。例如：

- 原书算法 3.21 及相邻叙述中的 Cauchy 步缩放和预测增益分支存在记录在案的记法疑点。第 11 章直接从二次模型计算预测下降。
- 原书附录式 (B.3) 的外积转置有维度疑问。教材 SVD 表达式保持 $u_j^T y$ 是标量、v_j 是参数方向的维度一致性。
- 原书的图号缺口、个别下标及绝对值符号问题保留在译本的检查报告里，不在教材里重新编号成“原书修订版”。

完整历史记录保存在 references/nonlinear-least-squares/source/Full_Check_Report.md。当前单独提供的中文版 PDF 与工程自带 PDF 字节不同，归档 README 和 SHA256 清单分别标识；本次不宣称已重新逐段校对两份译本。

D.4 初版的验证与边界

Notebook 使用独立 IPython 进程逐章执行，保存真实文本和图形输出；本次受执行环境限制，没有使用需要本地通信端口的 Jupyter 内核协议。标准 Notebook 可在本机 Jupyter 中打开，构建脚本也可在普通环境使用同一独立进程方式复现。

测试验证了有限差分、白化方向、解与参考结果、拒绝/停止和递推等价性；它们不构成所有输入上的正确性证明。工程案例均使用明确标注的合成数据，尚未验证真实屏幕或传感器。附录 C的完整混合/割线求解器及大规模稀疏实现留作后续扩展。

D.5 练习与解答

练习：引用一个原书公式前至少核对什么？解答：残差符号、向量行列约定、目标中的 1/2 系数、参数单位、适用假设和原书上下文；单独抄一个等式往往会丢失最关键的条件。